

SUPPORT DE COURS • DATA SCIENCE & ML ENGINEERING

Concevoir des API pour la Data Science

Python, FastAPI et mise à disposition de modèles de Machine Learning

Du fichier `model.joblib` au service `POST /predict`

Table des matières

Introduction	1
I Comprendre les API	3
1 Qu'est-ce qu'une API ?	4
1.1 Le problème de départ	4
1.2 Définition simple, définition technique	4
1.3 Pourquoi les API existent	5
1.4 Client et serveur	5
1.5 Application, serveur, API, base de données	5
1.6 Pourquoi les API sont utiles en Data Science	6
1.7 Que se passe-t-il quand une application demande une prédiction ?	6
II HTTP et REST	8
2 Les bases de HTTP	9
2.1 Requête et réponse	9
2.2 Anatomie d'une URL	9
2.3 Les méthodes HTTP	9
2.4 Les composants d'une requête	10
2.5 La réponse et les codes de statut	10
2.6 Ce qui se passe réellement entre le JSON et la prédiction	11
3 REST et API REST	13
3.1 Qu'est-ce que REST ?	13
3.2 Les idées clés	13
3.3 Mauvaise conception et bonne conception	13

III Premiers pas avec FastAPI	15
4 Pourquoi FastAPI ?	16
4.1 Pourquoi FastAPI existe	16
4.2 Vue d'ensemble : qui fait quoi ?	16
4.3 Les briques de base, avec du code minimal	17
5 Votre première API, ligne par ligne	18
5.1 Préparer l'environnement	18
5.2 Le code	18
5.3 Explication ligne par ligne	18
5.4 Lancer le serveur	19
5.5 Tester l'API	19
5.6 Ajoutons deux routes	20
IV Données, JSON et Pydantic	22
6 Comment une API reçoit des données	23
6.1 Paramètres de chemin (<i>path</i>)	23
6.2 Paramètres de requête (<i>query</i>)	23
6.3 Corps de la requête (<i>body</i>)	23
7 Pydantic : décrire et valider les données	25
7.1 Le rôle de Pydantic	25
7.2 Premier modèle	25
7.3 Utiliser le modèle dans une route	25
7.4 Ce qui se passe quand l'utilisateur envoie un JSON	26
7.5 Contraintes, optionnels et valeurs restreintes	26
7.6 Valider, c'est aussi protéger le modèle	27
7.7 Modèles de réponse	27
V FastAPI + Machine Learning	29
8 Anatomie d'une prédiction par API	30
8.1 Entraînement et inférence	30

8.2	Le chemin complet d'une requête de prédiction	30
9	Votre premier /predict	32
9.1	Le modèle : un point de départ	32
9.2	L'API	32
9.3	Explication détaillée	33
9.4	Tester	33
9.5	Les limites de cette première version	33
VI	Modèles .pkl et .joblib	35
10	Sauvegarder et charger un modèle	36
10.1	Pourquoi sauvegarder un modèle ?	36
10.2	Que contient le fichier ?	36
10.3	pickle ou joblib ?	36
10.4	Deux dangers à connaître	37
10.5	Sauvegarder aussi les métadonnées	37
11	Quand et comment charger le modèle	39
11.1	La mauvaise idée : charger à chaque requête	39
11.2	La bonne idée : charger une seule fois	39
VII	Prétraitement et Pipelines	42
12	Le prétraitement fait partie du modèle	43
12.1	Le décalage entre entraînement et service	43
12.2	Les pièges classiques	43
12.3	Démonstration chiffrée	44
12.4	Deux architectures	44
13	Un Pipeline complet, de l'entraînement à l'API	45
13.1	Construire le Pipeline	45
13.2	Pourquoi c'est idéal pour une API	46
13.3	Recharger le Pipeline dans FastAPI	46
13.4	Ce qui peut encore mal se passer	46

VIII	Architecture d'une API ML	49
14	D'un seul fichier à une architecture propre	50
14.1	Étape 1 : tout dans <code>main.py</code>	50
14.2	Étape 2 : séparer les responsabilités	50
14.3	Étape 3 : routes, services, configuration	51
14.4	Un exemple de route fine	51
15	Faire évoluer l'API : versions, plusieurs modèles, base de données	53
15.1	Versionner une API	53
15.2	Plusieurs modèles : une API, plusieurs endpoints, plusieurs services	53
15.3	Deux modèles dans une même API	54
15.4	API et base de données	54
IX	Tests, erreurs et documentation	57
16	Gérer les erreurs proprement	58
16.1	Les codes de statut à connaître	58
16.2	Les erreurs de validation (422)	58
16.3	Lever une erreur volontairement : <code>HTTPException</code>	59
16.4	Les erreurs du modèle	59
17	Tester une API à la main	61
17.1	<code>curl</code>	61
17.2	Python avec <code>requests</code>	61
18	Tests automatisés avec <code>pytest</code>	63
18.1	Mise en place	63
18.2	Les quatre types de tests essentiels	63
18.3	Remplacer le modèle par un faux	64
19	Documentation automatique	65
19.1	OpenAPI, Swagger UI, ReDoc	65
19.2	Pourquoi la documentation est importante	65
19.3	Documenter correctement un endpoint de prédiction	65

X	API et applications clientes	68
20	Comment les Data Scientists utilisent réellement les API	69
20.1	Scénario 1 : du notebook à l'API	69
20.2	Scénario 2 : une application web utilise le modèle	69
20.3	Scénario 3 : une application mobile appelle l'API	70
20.4	Scénario 4 : une autre équipe utilise l'API sans connaître le modèle	70
21	L'API comme contrat	71
21.1	Ce que le contrat précise	71
21.2	Ce qui reste libre	71
22	Un client Python pour consommer l'API	73
22.1	Le code	73
22.2	Le rôle de ce client	74
XI	Sécurité et bonnes pratiques	76
23	Les bases de la sécurité d'une API	77
23.1	API publique ou privée	77
23.2	Authentification et autorisation	77
23.3	Clés d'API	77
23.4	JWT (JSON Web Token)	78
23.5	CORS	78
23.6	Autres bonnes pratiques	78
24	Configuration et journalisation	80
24.1	Variables d'environnement et fichier <code>.env</code>	80
24.2	Une configuration propre avec <code>pydantic-settings</code>	80
24.3	Journalisation (<i>logging</i>)	81
25	Performance et conception	83
25.1	Latence et temps de réponse	83
25.2	Charger une fois, prédire souvent	83
25.3	Concurrence	83
25.4	Prédiction par lot (<i>batch</i>)	83

25.5 CPU ou GPU	84
XII Préparation à la production	85
26 Du développement à la production : le chemin	86
26.1 Le parcours	86
26.2 Développement et production	86
26.3 Les briques à connaître	86
XIII Projet final complet	89
27 Le projet <code>ml_prediction_api</code>	90
27.1 Le scénario	90
27.2 Arborescence	90
27.3 Rôle de chaque fichier	91
27.4 L'entraînement (côté Data Scientist)	91
27.5 La configuration	93
27.6 Le contrat : <code>schemas.py</code>	93
27.7 Le service ML : <code>model_service.py</code>	95
27.8 Sécurité et routes	97
27.9 L'application : <code>main.py</code>	98
27.10 Les tests	100
27.11 Lancer le projet	101
27.12 Réponse type	101
Conclusion	103
Checklist finale	104
Aide-mémoire	105

Introduction

Vous savez entraîner un modèle : nettoyer des données, comparer des algorithmes, régler des hyperparamètres, évaluer avec précision, rappel et F1-score, puis sauvegarder le résultat dans un fichier `.joblib`. Ce travail est essentiel, mais il laisse une question sans réponse :

La question de départ

« J'ai déjà un modèle entraîné et sauvegardé. Comment une *autre* application peut-elle lui envoyer des données et obtenir une prédiction ? »

Ce cours répond à cette question de bout en bout. Il ne s'agit *pas* d'un tutoriel FastAPI de plus : l'objectif est de comprendre *pourquoi* les API existent, *comment* une requête voyage du client jusqu'au modèle, et *pourquoi* chaque brique d'un service de prédiction propre existe.

Objectifs pédagogiques

À la fin du cours, vous serez capable de :

- expliquer précisément ce qu'est une API, ce qu'est HTTP et ce que signifie REST ;
- construire une API FastAPI de zéro, en comprenant chaque ligne ;
- définir un schéma d'entrée avec Pydantic et expliquer la validation qui en découle ;
- charger un modèle (ou mieux, un **Pipeline**) *une seule fois* et l'exposer via `POST /predict` ;
- expliquer pourquoi le prétraitement doit voyager avec le modèle ;
- gérer les erreurs, documenter, tester et sécuriser une API de prédiction ;
- organiser un projet de façon professionnelle et comprendre le chemin qui mène à la production.

Le fil conducteur du cours est le suivant :



Comment lire ce cours

Chaque notion suit la même progression : *concept*, *analogie* si utile, *exemple minimal*, *explication du code*, *exemple réaliste*, puis *exercice*. Ne sautez pas les exemples minimaux : ils installent le vocabulaire sur lequel reposent les chapitres suivants.

Les encadrés ont un rôle précis :

À retenir

Une idée à mémoriser : si vous ne devez retenir qu'une chose d'une section, c'est ici.

Attention

Un piège classique, une erreur fréquente ou un danger en pratique.

Question de compréhension 0.1

Une question pour vérifier que vous avez *compris*, pas seulement mémorisé. Essayez d'y répondre avant de continuer.

Les blocs de code Python ont un fond gris clair ; les **commandes à taper dans un terminal** ont un fond sombre :

```
$ python --version
```

Prérequis et environnement

Vous avez besoin de Python 3.10 ou plus, de `pip` et d'un éditeur de code. Nous travaillerons dans un environnement virtuel (voir le chapitre 5). Les exercices sont numérotés de façon continue ; leur *niveau* (de 1 à 10) indique la progression visée : API GET simple, POST avec JSON, Pydantic, chargement d'un modèle, `/predict`, Pipeline, erreurs, tests, architecture propre, projet final. Les corrections se trouvent dans une section distincte intitulée « Correction », après les exercices : essayez avant de regarder.

Production et conception sont deux sujets

Ce cours vous apprend à *construire* une bonne API, en local. La *mise en production* (Docker, serveurs, HTTPS, monitoring) est présentée au chapitre 26 comme une préparation mentale, pas comme un cours DevOps complet.

PARTIE I

Comprendre les API

Qu'est-ce qu'une API ?

1.1 Le problème de départ

Imaginez le scénario suivant. Dans un notebook, vous avez entraîné un modèle qui estime si un client obtiendra un prêt. Il fonctionne : $F_1 = 0,91$ sur le jeu de test. Une équipe mobile veut maintenant afficher cette décision dans son application. Plusieurs obstacles apparaissent aussitôt :

- l'application mobile est écrite en Kotlin ou Swift, pas en Python ;
- elle ne peut pas ouvrir votre notebook, ni importer `scikit-learn` ;
- vous ne voulez pas envoyer votre fichier de modèle à tout le monde ;
- vous voulez pouvoir améliorer le modèle *sans* demander à l'équipe mobile de modifier son code.

Une API est la réponse standard à ce problème : elle place le modèle derrière une *porte de communication* bien définie.

1.2 Définition simple, définition technique

Analogie : le restaurant

Vous (le *client*) ne rentrez pas dans la cuisine. Vous passez commande à un serveur en suivant le menu ; la cuisine prépare le plat et le serveur vous l'apporte. Le menu fixe ce qui est possible, et la cuisine peut changer de cuisinier ou de recette sans que vous ayez à le savoir. Ici, le serveur joue le rôle de l'API et la cuisine celui du modèle.

API — définition technique

Une **API** (*Application Programming Interface*) est un ensemble de règles qui permet à un programme d'utiliser les fonctionnalités d'un autre programme sans connaître son fonctionnement interne. Dans ce cours, nous parlons d'**API web** : le dialogue se fait par le réseau, avec le protocole HTTP, et les données sont échangées le plus souvent au format JSON.

Deux mots résument tout : *interface* (ce qu'on peut demander et comment) et *abstraction* (on ne voit pas ce qui se passe derrière).

1.3 Pourquoi les API existent

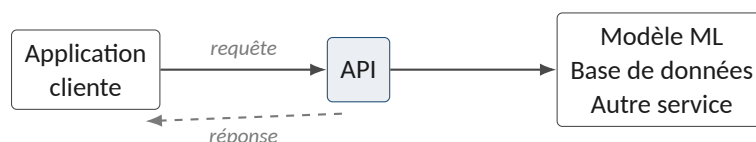
- **Découplage** : le client et le serveur évoluent indépendamment tant que l'interface ne change pas.
- **Indépendance du langage** : JSON et HTTP sont compris par Python, JavaScript, Kotlin, Java, etc.
- **Réutilisation** : un même service peut être utilisé par un site web, une application mobile et un script d'analyse.
- **Contrôle** : seul le serveur voit les données et le code ; on peut y appliquer authentification, validation et journalisation.
- **Centralisation** : une seule version du modèle est déployée, et tout le monde bénéficie de la mise à jour.

1.4 Client et serveur

Le modèle **client-serveur** est le socle de tout ce qui suit.

- Le **client** est le programme qui *demande* (navigateur, application mobile, script Python, autre API).
- Le **serveur** est le programme qui *répond* : il tourne en continu, attend des demandes et les traite.

Un point d'attention : « serveur » désigne un *programme*, pas forcément une machine puissante. Sur votre ordinateur, un serveur FastAPI est simplement un processus Python qui écoute sur un port.



1.5 Application, serveur, API, base de données

Ces quatre termes sont souvent confondus. Le tableau suivant les sépare.

Terme	Rôle	Exemple
Application	Ce que l'utilisateur voit et manipule	Application mobile d'une banque
Serveur	Programme qui écoute et répond aux demandes	Processus Uvicorn
API	Contrat et point d'entrée : ce qu'on peut demander, sous quelle forme	POST /predict
Base de données	Stockage durable et organisé des données	PostgreSQL, SQLite

Un même programme peut jouer plusieurs rôles : un serveur FastAPI *héberge* une API ; cette API peut *lire* une base de données et *appeler* un modèle. L'application, elle, ne parle qu'à l'API.

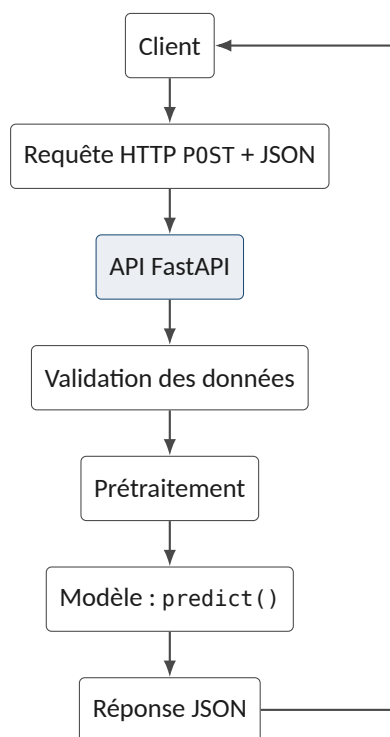
1.6 Pourquoi les API sont utiles en Data Science

Un modèle n'a de valeur que s'il est utilisé. L'API est le pont entre le monde du Data Scientist (notebooks, Python, fichiers) et celui du produit (applications, équipes, utilisateurs). Elle permet de :

- rendre un modèle accessible à n'importe quelle application ;
- séparer clairement *entraînement* (hors ligne) et *inférence* (en ligne) ;
- imposer un format d'entrée strict (types, bornes, champs obligatoires) ;
- surveiller l'usage et la qualité des prédictions.

1.7 Que se passe-t-il quand une application demande une prédiction ?

Voici le déroulé complet, que nous détaillerons chapitre après chapitre.



1. Le client envoie une requête HTTP contenant des données au format JSON.
2. Le serveur reçoit la requête et l'aiguille vers la bonne fonction.
3. Les données sont *validées* : types, bornes, champs manquants.
4. Elles sont transformées dans la forme attendue par le modèle.
5. Le modèle calcule une prédiction.
6. Le résultat est mis en forme en JSON et renvoyé au client.

À retenir

Une API de prédiction ne « contient » pas le modèle pour le montrer : elle en est la *porte d'entrée contrôlée*. Le client envoie des données, reçoit un résultat, et ne voit jamais le code Python.

Question de compréhension 1.1

Quelle différence faites-vous entre le serveur (programme qui écoute) et l'API (contrat qu'il expose) ?

Exercices

Exercice 1 • Niveau 1 — Dessiner l'architecture

Objectif : vérifier que vous distinguez les rôles. **Consignes** : pour une application mobile de prêt bancaire utilisant votre modèle, dessinez le schéma client, API, modèle, base de données, puis écrivez en une phrase le rôle de chaque bloc. **Résultat attendu** : un schéma et quatre phrases.

Correction

Correction — Dessiner l'architecture

Application mobile → API → service de modèle → base de données. L'application affiche et collecte des données ; l'API reçoit, valide et répond ; le modèle calcule la prédiction ; la base de données conserve l'historique. Le point clé : l'application ne parle qu'à l'API.

PARTIE II

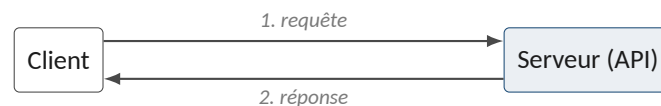
HTTP et REST

Les bases de HTTP

HTTP (*HyperText Transfer Protocol*) est le langage commun du Web. Une API web est avant tout une *manière de parler HTTP*. Avant FastAPI, il faut comprendre cette conversation.

2.1 Requête et réponse

HTTP fonctionne en *questions-réponses* : le client envoie une **requête**, le serveur renvoie une **réponse**. Chaque échange est indépendant ; le serveur ne « se souvient » pas de la requête précédente (nous y reviendrons avec REST).



2.2 Anatomie d'une URL

```
http://127.0.0.1:8000/api/v1/predict?model=risk
 \____/ \_____/ \____/ \_____/ \_____/
schéma   hôte   port   chemin   paramètres
```

- **Schéma** : le protocole (`http` ou `https`).
- **Hôte** : la machine ; `127.0.0.1` (ou `localhost`) désigne *votre propre ordinateur*.
- **Port** : la « porte » sur laquelle le programme écoute (8000 par défaut avec Uvicorn).
- **Chemin** (*path*) : la ressource visée.
- **Paramètres** (*query*) : informations optionnelles après le `?`.

Un **endpoint** est le couple *méthode + chemin* auquel l'API répond, par exemple `POST /api/v1/predict`. Une **route** est la manière dont le framework associe ce chemin à une fonction Python. Dans la pratique, on emploie les deux mots presque indifféremment ; techniquement, la route est la déclaration côté code, l'endpoint est ce que voit le client.

2.3 Les méthodes HTTP

La méthode exprime l'*intention* de la requête.

Méthode	Intention	Exemple
GET	Lire une ressource, sans la modifier	GET /patients/12
POST	Envoyer des données à traiter ou créer une ressource	POST /predict
PUT	Remplacer entièrement une ressource	PUT /patients/12
PATCH	Modifier partiellement une ressource	PATCH /patients/12
DELETE	Supprimer une ressource	DELETE /patients/12

Pourquoi POST pour une prédiction ?

Une prédiction demande d'envoyer des données structurées (parfois nombreuses ou sensibles). Avec **GET**, les données iraient dans l'URL : longueur limitée, valeurs visibles dans les journaux et l'historique, structure imbriquée difficile. **POST** transporte les données dans le *corps* (*body*) de la requête, en JSON, sans ces limites. De plus, **GET** doit rester « sans effet de bord » et mis en cache, ce qui ne correspond pas à un calcul déclenché par des données.

2.4 Les composants d'une requête

Voici une requête complète envoyée à une API de prédiction :

```
POST /predict HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json
Accept: application/json

{"age": 25, "income": 350000, "education": 3}
```

- **Ligne de requête** : méthode, chemin, version du protocole.
- **En-têtes** (*headers*) : métadonnées. **Content-Type** annonce le format du corps ; **Authorization** ou **X-API-Key** transportent l'authentification.
- **Ligne vide**, puis **corps** (*body*) : les données elles-mêmes.

Il existe trois façons de transmettre de l'information :

Type	Où ?	Usage typique
Path parameter	Dans le chemin : /patients/12	Identifier une ressource
Query parameter	Après ? : ?limit=10	Filtrer, paginer
Body	Dans le corps, en JSON	Données à traiter (prédiction)

2.5 La réponse et les codes de statut

La réponse contient un **code de statut**, des en-têtes et un corps :

```
HTTP/1.1 200 OK
Content-Type: application/json

{"prediction": 1}
```

Le code de statut résume le résultat en trois chiffres :

Famille	Signification et codes courants
2xx	Succès : 200 OK, 201 Created, 204 No Content
4xx	Erreur <i>du client</i> : 400 requête incorrecte, 401 non authentifié, 403 interdit, 404 introuvable, 422 données invalides
5xx	Erreur <i>du serveur</i> : 500 erreur interne, 503 service indisponible

2.6 Ce qui se passe réellement entre le JSON et la prédiction

Reprenons l'échange de l'introduction. Le client envoie :

```
{
  "age": 25,
  "income": 350000,
  "education": 3
}
```

et reçoit :

```
{
  "prediction": 1
}
```

Entre les deux, le serveur effectue au minimum les opérations suivantes :

1. il lit le corps de la requête (du texte) ;
2. il *déséréalise* ce texte JSON en objet Python (un dictionnaire, puis un objet validé) ;
3. il vérifie les types et les contraintes ;
4. il place les valeurs dans la structure attendue par le modèle (ordre des colonnes!) ;
5. il appelle `model.predict(...)`, qui renvoie par exemple un tableau NumPy ;
6. il convertit ce résultat en types Python simples puis le *séréalise* en JSON.

Attention

Un tableau NumPy `array([1])` ou un `numpy.int64` n'est pas du JSON. Il faut le convertir (par exemple avec `int(...)`) avant de répondre : c'est une source d'erreurs classique en débutant.

Question de compréhension 2.1

Pourquoi le serveur doit-il vérifier le format des données *avant* d'appeler `predict()` ?

REST et API REST

3.1 Qu'est-ce que REST ?

REST (*REpresentational State Transfer*) n'est pas une technologie mais un **style d'architecture** pour concevoir des API sur HTTP. Une API qui suit ces conventions est dite *RESTful*. Son avantage : tout développeur sait, sans lire de long manuel, comment l'utiliser.

3.2 Les idées clés

- **Ressources** : on expose des *choses* (patients, modèles, prédictions), identifiées par des URL. Les noms sont des *noms*, pas des verbes.
- **Méthodes HTTP** : le verbe est dans la méthode (GET, POST...), pas dans l'URL.
- **Stateless** (sans état) : chaque requête contient tout ce qu'il faut pour être traitée ; le serveur ne s'appuie pas sur la requête précédente. Cela facilite le passage à l'échelle : n'importe quelle copie du serveur peut répondre.
- **Représentations** : on échange le plus souvent du JSON.
- **Codes de statut** cohérents pour signaler le résultat.

3.3 Mauvaise conception et bonne conception

À éviter	Préférable
GET /getPatientById?id=12	GET /patients/12
POST /deletePatient	DELETE /patients/12
GET /predict?age=25&income=350000	POST /predict avec corps JSON
POST /doEverything	Un endpoint par fonction claire
Toujours 200, même en cas d'erreur	Codes adaptés : 422, 404, 500

Pourquoi /api/v1/predict plutôt que /getPrediction ?

- /api signale une API, v1 sa version (voir chapitre 15) ;
- predict décrit la *capacité* offerte ; l'action est portée par POST ;
- le nom est court, stable et lisible.

Une prédiction est-elle une ressource ?

Cas particulier courant en ML : /predict est un *service de calcul*, pas une ressource stockée. Cette convention (verbe « predict » dans l'URL, méthode POST) est largement acceptée. Si l'on enregistre chaque prédiction, on peut aussi exposer une ressource : POST

/predictions crée une prédiction, GET /predictions/42 la relit.

À retenir

Un bon design REST se reconnaît à ceci : on devine l'URL, la méthode et le code de retour sans consulter la documentation.

Question de compréhension 3.1

Pourquoi /predict/1 (avec un identifiant) serait-il étrange pour un endpoint qui calcule une prédiction à partir de données envoyées ?

Exercices

Exercice 2 • Niveau 0 — Lire HTTP et concevoir des routes

Objectif : manipuler le vocabulaire HTTP/REST. **Consignes** : (a) pour la requête `POST /api/v1/predict?debug=true`, identifiez méthode, chemin, paramètre de requête et corps ; (b) corrigez les routes `POST /createUser`, `GET /deleteUser?id=3`, `GET /getAllModels` ; (c) quel code de statut pour des données invalides ? pour un modèle non chargé ? **Résultat attendu** : réponses écrites.

Correction

Correction — Lire HTTP et concevoir des routes

(a) Méthode : `POST` ; chemin : `/api/v1/predict` ; paramètre de requête : `debug=true` ; corps : le JSON des données, placé après les en-têtes. (b) `POST /users`, `DELETE /users/3`, `GET /models`. (c) 422 pour des données invalides ; 503 pour un modèle indisponible (ou 500 pour une erreur interne inattendue).

PARTIE III

Premiers pas avec FastAPI

Pourquoi FastAPI ?

Vous savez désormais ce qu'est une API et comment HTTP transporte les requêtes. Reste à les *implémenter*. Un framework web fait ce travail répétitif à votre place : écouter le réseau, lire les requêtes HTTP, appeler votre fonction et fabriquer la réponse.

4.1 Pourquoi FastAPI existe

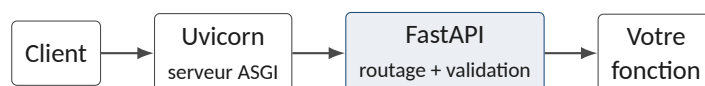
Avant FastAPI, les frameworks Python dominants étaient Flask (léger, mais sans validation ni documentation intégrées) et Django (complet, mais plus lourd). FastAPI combine deux bibliothèques existantes :

- **Starlette** pour la partie web (routage, requêtes, réponses, asynchrone) ;
- **Pydantic** pour la partie données (validation et conversion à partir des annotations de types Python).

Avantages qui comptent en Data Science :

- les *annotations de types* suffisent à décrire et valider les données (`age: int`) ;
- une **documentation interactive** est générée automatiquement ;
- de bonnes performances, grâce à l'asynchrone et à Starlette ;
- peu de code répétitif : une fonction Python devient un endpoint.

4.2 Vue d'ensemble : qui fait quoi ?



- **Uvicorn** est le *serveur* : il écoute le port, reçoit les octets HTTP, les passe à l'application, renvoie la réponse. Il suit la norme **ASGI**, l'interface standard entre serveurs et applications asynchrones.
- **FastAPI** est l'*application* : elle trouve la fonction qui correspond à la méthode et au chemin, valide les données, appelle votre fonction, sérialise le résultat.
- **Votre fonction** contient votre logique (ici : appeler un modèle).

À retenir

FastAPI *n'est pas* un serveur : il a besoin d'Uvicorn (ou équivalent) pour tourner. Ce partage des rôles revient souvent dans la littérature sur le déploiement.

4.3 Les briques de base, avec du code minimal

Voici chaque élément du vocabulaire FastAPI, en quelques lignes. Nous les détaillerons dans le chapitre suivant.

```
from fastapi import FastAPI

app = FastAPI()           # 1. crée l'application

@app.get("/hello")        # 2. route : GET /hello
def hello():
    return {"message": "bonjour"} # 3. dict Python -> JSON

@app.post("/echo")        # 4. route : POST /echo
def echo(data: dict):    # 5. le corps JSON arrive dans "data"
    return data
```

Notion	Rôle
FastAPI()	Crée l'objet application qui contient toutes les routes
@app.get(...)	Décorateur : associe une fonction à GET + un chemin
@app.post(...)	Idem pour POST
Paramètres	Valeurs lues dans le chemin, la requête ou le corps, selon l'annotation
Validation	Vérification automatique des types et contraintes
Désérialisation	JSON (texte) → objets Python
Sérialisation	Objets Python → JSON (texte)
OpenAPI	Description standardisée de l'API (fichier JSON), générée automatiquement
Swagger UI	Interface web interactive qui lit cette description (/docs)

Question de compréhension 4.1

Si FastAPI ne gère pas lui-même le réseau, quel composant s'en charge, et que reçoit-il en entrée ?

Votre première API, ligne par ligne

Nous construisons une API *sans Machine Learning*. Le but est de comprendre le mécanisme, sans rien considérer comme évident.

5.1 Préparer l'environnement

Un **environnement virtuel** isole les bibliothèques du projet. On évite ainsi les conflits entre versions d'un projet à l'autre.

```
$ mkdir ml_api_demo && cd ml_api_demo
$ python -m venv .venv
$ source .venv/bin/activate          # Windows : .venv\Scripts\activate
$ pip install fastapi "uvicorn[standard]"
```

5.2 Le code

Créez un fichier `main.py` :

main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "API opérationnelle"}
```

5.3 Explication ligne par ligne

from fastapi import FastAPI

Importe la classe `FastAPI` depuis la bibliothèque installée.

app = FastAPI()

Crée une *instance* de l'application. `app` est l'objet central : il contiendra le tableau de correspondance « méthode + chemin » → « fonction ». Son nom est une convention ; c'est lui qu'Uvicorn va chercher.

@app.get("/")

Un *décorateur* : il enregistre la fonction qui suit dans l'application, pour les requêtes `GET` sur le

chemin `/`. Cette ligne *ne s'exécute pas* à chaque requête : elle déclare la route au démarrage.

`/`

Le chemin « racine » : `http://127.0.0.1:8000/`. C'est la *route*.

def home():

La fonction appelée quand une requête correspond. Son nom importe peu pour le client ; il apparaît dans la documentation.

return {...}

Un dictionnaire Python. FastAPI le *séréalise* en JSON, ajoute l'en-tête `Content-Type: application/json` et le code 200.

5.4 Lancer le serveur

```
$ uvicorn main:app --reload
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

- `main:app` signifie « dans le fichier `main.py`, prends la variable `app` » ;
- `--reload` relance automatiquement le serveur à chaque modification du code (développement uniquement) ;
- l'adresse `127.0.0.1:8000` est votre machine, port 8000.

5.5 Tester l'API

Navigateur

Ouvrez `http://127.0.0.1:8000/`. Le navigateur envoie un `GET` et affiche le JSON reçu.

curl

```
$ curl -i http://127.0.0.1:8000/
HTTP/1.1 200 OK
content-type: application/json

{"message":"API opérationnelle"}
```

Swagger UI

Ouvrez `http://127.0.0.1:8000/docs`. Vous y trouvez la liste des endpoints, leurs paramètres, et un bouton « Try it out » pour envoyer des requêtes sans écrire de code. La même description, au format brut (OpenAPI), est servie sur `/openapi.json`, et une présentation alternative sur `/redoc`.

Erreurs fréquentes

- **Error loading ASGI app** : le nom du fichier ou de la variable est faux (`main:app`).
- **Address already in use** : un autre serveur occupe déjà le port ; utilisez `--port 8001`.
- **404 Not Found** : le chemin ou la méthode ne correspond à aucune route déclarée.
- Après l'installation, rien ne marche : vérifiez que l'environnement virtuel est *activé*.

5.6 Ajoutons deux routes

```
@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/hello/{name}")
def hello(name: str):
    return {"message": f"Bonjour {name}"}
```

`{name}` dans le chemin est un *paramètre de chemin* : sa valeur est transmise à l'argument `name`. Nous les étudions au chapitre suivant. La route `/health` sert de *sonde de santé* : les outils de supervision l'interrogent pour savoir si le service répond.

À retenir

`@app.get("/")` signifie, dans le fonctionnement HTTP : « quand une requête GET arrive sur le chemin `/`, appelle cette fonction et transforme son résultat en réponse JSON ».

Question de compréhension 5.1

Que se passe-t-il si vous envoyez `POST /` à cette API ? Pourquoi ?

Exercices

Exercice 3 • Niveau 1 — Une API GET simple

Objectif : construire et lancer votre première API. **Consignes** : créez une API avec (a) `GET /` qui renvoie un message, (b) `GET /health`, (c) `GET /square/{n}` qui renvoie le carré d'un entier. Testez dans le navigateur, avec curl et avec Swagger. **Résultat attendu** : `GET /square/4` renvoie `{"n": 4, "square": 16}`.

Correction

Correction — Une API GET simple

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "API opérationnelle"}

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/square/{n}")
def square(n: int):
    return {"n": n, "square": n * n}
```

Grâce à l'annotation `n: int`, GET `/square/abc` renvoie automatiquement une erreur 422.

PARTIE IV

Données, JSON et Pydantic

Comment une API reçoit des données

Une API de prédiction vit de données entrantes. FastAPI sait en lire de trois endroits, selon la façon dont vous déclarez les arguments de la fonction.

6.1 Paramètres de chemin (*path*)

```
@app.get("/patients/{patient_id}")
def get_patient(patient_id: int):
    return {"patient_id": patient_id}
```

Pour GET `/patients/12`, FastAPI extrait 12 du chemin, le *convertit* en entier grâce à l'annotation, puis appelle la fonction. Si le client envoie GET `/patients/abc`, la fonction n'est **jamais appelée** : l'API répond 422 avec une explication.

Usage typique : *identifier une ressource*.

6.2 Paramètres de requête (*query*)

Tout argument qui n'apparaît pas dans le chemin et n'est pas un modèle Pydantic est lu dans la requête, après le `?`.

```
@app.get("/patients")
def list_patients(limit: int = 10, skip: int = 0):
    return {"limit": limit, "skip": skip}
```

- GET `/patients` → valeurs par défaut : `limit=10, skip=0`;
- GET `/patients?limit=5&skip=20` → valeurs fournies;
- un argument *sans* valeur par défaut est obligatoire.

Usage typique : *filtrer, trier, paginer*.

6.3 Corps de la requête (*body*)

Pour envoyer des données structurées, on utilise le corps, au format JSON. Première tentative, avec un simple dictionnaire :

```
@app.post("/predict")
def predict(data: dict):
    return {"received": data}
```

Cela fonctionne, mais *rien* n'est vérifié : **age** peut être une chaîne, un champ peut manquer, une valeur peut être absurde. Pour un modèle de Machine Learning, c'est dangereux : l'erreur surviendra plus tard, dans `predict()`, avec un message incompréhensible. C'est pour cela qu'existe Pydantic.

Où ?	Déclaration	Exemple d'usage
Chemin	Nom entre accolades dans la route	<code>/patients/12</code>
Requête	Argument simple avec ou sans défaut	<code>?limit=10</code>
Corps	Argument de type modèle Pydantic	JSON de prédiction

Question de compréhension 6.1

Dans `@app.get("/items/{item_id}")` avec `def f(item_id: int, q: str = "")`, lequel des deux arguments est un paramètre de chemin et lequel est un paramètre de requête ?

Pydantic : décrire et valider les données

7.1 Le rôle de Pydantic

Pydantic

Pydantic est une bibliothèque qui transforme des données brutes (dictionnaire issu d'un JSON) en *objets Python typés*, en les validant. Si les données ne respectent pas les types ou contraintes déclarés, elle lève une erreur détaillée.

Dans une API, un modèle Pydantic joue trois rôles à la fois : **schéma** (il décrit les champs attendus), **validateur** (il refuse les mauvaises données) et **documentation** (il alimente Swagger).

7.2 Premier modèle

schemas.py

```
from pydantic import BaseModel

class PatientData(BaseModel):
    age: int
    weight: float
    height: float
```

- `BaseModel` est la classe parente qui apporte la validation ;
- chaque *attribut annoté* est un champ du JSON attendu ;
- tous les champs sont obligatoires, sauf s'ils ont une valeur par défaut.

7.3 Utiliser le modèle dans une route

```
from fastapi import FastAPI
from schemas import PatientData

app = FastAPI()

@app.post("/patients")
def create_patient(patient: PatientData):
    bmi = patient.weight / (patient.height / 100) ** 2
    return {"age": patient.age, "bmi": round(bmi, 1)}
```

Parce que `patient` est annoté avec un modèle Pydantic, FastAPI comprend que ses données viennent du *corps* de la requête.

7.4 Ce qui se passe quand l'utilisateur envoie un JSON

```
{"age": 10, "weight": 32.5, "height": 135}
```



1. FastAPI lit le corps et **déséréalise** le JSON en dictionnaire Python.
2. Pydantic parcourt les champs déclarés : présence, type, contraintes.
3. Il **convertit** si c'est sans ambiguïté (la chaîne "10" devient l'entier 10).
4. Il construit l'objet `PatientData`, que votre fonction reçoit.
5. En cas de problème, la fonction n'est *pas appelée* : FastAPI renvoie 422.

Que voit le client en cas d'erreur ?

Avec `\\{"age": "dix", "weight": 32.5\\}` :

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["body", "age"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "dix"
    },
    {
      "type": "missing",
      "loc": ["body", "height"],
      "msg": "Field required"
    }
  ]
}
```

Chaque erreur indique *où* (`loc`), *quoi* (`type`) et *pourquoi* (`msg`). Un développeur frontend peut corriger sa requête sans lire votre code.

Conversion silencieuse

Pydantic accepte "10" pour un `int`, et 10.5 est refusé pour un `int` (partie décimale). Cette tolérance est pratique, mais pensez-y : un client qui envoie `"age": "25"` ne provoque pas d'erreur. Si vous voulez refuser, utilisez le mode strict (`ConfigDict(strict=True)`).

7.5 Contraintes, optionnels et valeurs restreintes

```
from typing import Literal
from pydantic import BaseModel, Field
```

```
class LoanRequest(BaseModel):
    age: int = Field(..., ge=18, le=100)          # obligatoire, entre 18 et 100
    income: float = Field(..., ge=0)              # positif
    education: Literal["secondaire", "licence", "master"]
    years_employed: float | None = None          # optionnel (peut manquer)
    debt_ratio: float = Field(0.0, ge=0, le=1)    # valeur par défaut
```

- `Field(..., ge=18)` : champ *obligatoire* (...) avec contrainte « supérieur ou égal à 18 » ;
- `Literal[...]` : seules ces valeurs exactes sont acceptées, idéal pour les *catégories* d'un modèle (voir chapitre 13) ;
- `float | None = None` : champ optionnel, utile si le modèle sait gérer les valeurs manquantes ;
- une valeur par défaut rend le champ facultatif.

7.6 Valider, c'est aussi protéger le modèle

Un modèle ne sait pas dire « cette donnée est absurde » : `age=-5` ou `income=10**15` produiront une prédiction sans erreur, mais sans signification. La validation Pydantic est la *première ligne de défense* ; elle bloque les entrées invalides avant qu'elles n'atteignent le modèle.

7.7 Modèles de réponse

On peut aussi décrire la *sortie* :

```
class PredictionResponse(BaseModel):
    prediction: int
    probability: float

@app.post("/predict", response_model=PredictionResponse)
def predict(data: LoanRequest):
    ...
```

`response_model` garantit que la réponse respecte le schéma (filtrage des champs superflus, conversion de types) et documente le contrat de sortie dans Swagger.

À retenir

Pydantic transforme un JSON *non fiable* en objet Python *garanti* : types corrects, valeurs dans les bornes, champs présents. Tout le code qui suit peut s'appuyer sur ces garanties.

Question de compréhension 7.1

Que se passe-t-il, pas à pas, quand un client envoie `\{"age": 10, "weight": 32.5\}` (sans `height`) à la route `/patients` ? Votre fonction est-elle exécutée ?

Exercices

Exercice 4 • Niveau 2 — POST avec JSON

Objectif : recevoir un corps JSON. **Consignes** : créez POST /echo qui reçoit un dictionnaire et le renvoie avec un champ "received": true. Testez avec curl. **Résultat attendu** : la même donnée renvoyée, enrichie.

Exercice 5 • Niveau 3 — Ajouter Pydantic

Objectif : valider des entrées. **Consignes** : créez POST /bmi qui reçoit weight (kg, > 0) et height (cm, > 0) et renvoie l'IMC. Testez des valeurs valides, négatives et manquantes. **Résultat attendu** : 200 pour un cas valide, 422 pour les autres.

Correction

Correction — POST avec JSON

```
@app.post("/echo")
def echo(data: dict):
    return {**data, "received": True}
```

```
$ curl -X POST http://127.0.0.1:8000/echo \
-H "Content-Type: application/json" -d '{"a": 1}'
{"a":1,"received":true}
```

Correction — Ajouter Pydantic

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class BmiInput(BaseModel):
    weight: float = Field(..., gt=0)
    height: float = Field(..., gt=0)

@app.post("/bmi")
def bmi(data: BmiInput):
    value = data.weight / (data.height / 100) ** 2
    return {"bmi": round(value, 1)}
```

Un poids négatif ou une taille manquante ne déclenchent *jamaïs* la fonction : FastAPI renvoie 422.

PARTIE V

FastAPI + Machine Learning

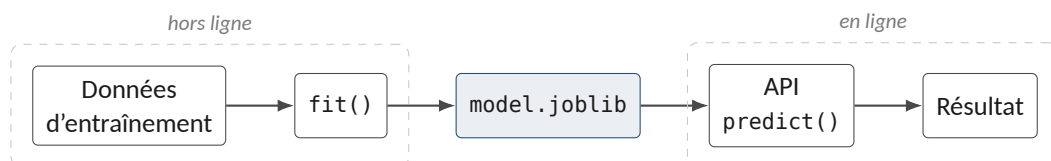
Anatomie d'une prédiction par API

Nous arrivons au cœur du cours. Tout ce qui précède (HTTP, JSON, FastAPI, Pydantic) sert un seul but : brancher un modèle sur le réseau *proprement*.

8.1 Entraînement et inférence

Ces deux phases n'ont ni le même rythme, ni les mêmes contraintes.

	Entraînement	Inférence
Objectif	Apprendre les paramètres	Appliquer le modèle à de nouvelles données
Fréquence	Occasionnelle (jours, semaines)	Continue (à chaque requête)
Appel typique	<code>model.fit(X, y)</code>	<code>model.predict(X)</code>
Coût	Élevé (minutes, heures)	Faible (millisecondes)
Où	Notebook, script d'entraînement	API en service
Données	Jeu complet, étiqueté	Une ou quelques lignes, sans étiquette

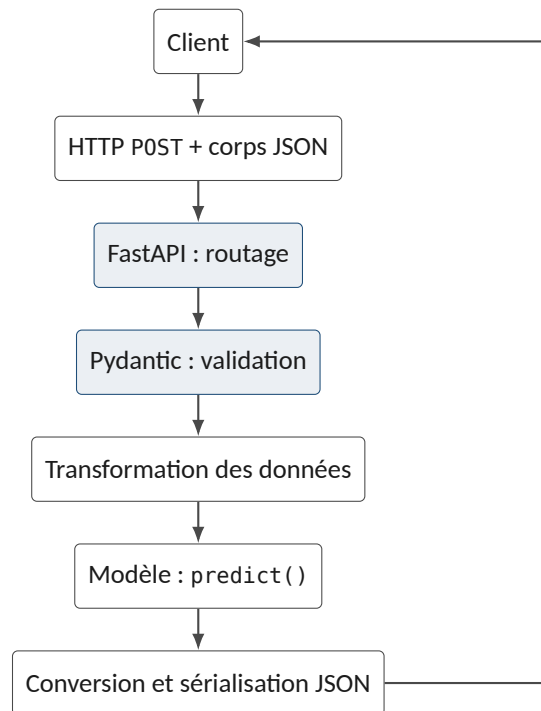


À retenir

Une API ne réentraîne *pas* le modèle à chaque requête : ce serait lent, coûteux et instable. Elle charge un modèle déjà entraîné et ne fait que de l'inférence.

8.2 Le chemin complet d'une requête de prédiction

Détaillons chaque étape du schéma vu au chapitre 1, et surtout *ce qui peut échouer* à chaque niveau.



Étape	Erreur possible	Code attendu
Réseau	Serveur éteint, mauvais port	Pas de réponse
Routage	Mauvaise URL ou méthode	404 / 405
Authentification	Clé absente ou invalide	401 / 403
Corps JSON	JSON mal formé	422
Validation	Champ manquant, mauvais type, hors bornes	422
Transformation	Catégorie inconnue, colonne manquante	422 ou 500
Modèle	Fichier absent, version incompatible	503 / 500
Sérialisation	Type NumPy non convertible	500

Question de compréhension 8.1

Que se passe-t-il entre l'envoi du JSON et l'appel à `model.predict()` ? Citez au moins quatre opérations et dites laquelle peut renvoyer une erreur 422.

Votre premier /predict

Construisons la version la plus simple d'une API de prédiction, en acceptant ses défauts : ils motivent les chapitres suivants.

9.1 Le modèle : un point de départ

Supposons un modèle simple entraîné sur trois variables : `age`, `income`, `education` (niveau de 1 à 5).

train_toy.py

```
import joblib
import numpy as np
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(0)
X = np.column_stack([
    rng.integers(18, 70, 500),           # age
    rng.normal(400_000, 150_000, 500).clip(50_000), # income
    rng.integers(1, 6, 500),             # education
])
y = ((X[:, 1] > 300_000) & (X[:, 2] >= 3)).astype(int)

model = RandomForestClassifier(n_estimators=50, random_state=0).fit(X, y)
joblib.dump(model, "model.joblib")
```

9.2 L'API

main.py

```
import joblib
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
model = joblib.load("model.joblib")

class Features(BaseModel):
    age: int
    income: float
    education: int

@app.post("/predict")
```

```
def predict(data: Features):  
    x = [[data.age, data.income, data.education]]  
    y = model.predict(x)  
    return {"prediction": int(y[0])}
```

9.3 Explication détaillée

@app.post("/predict")

Cette ligne *déclare* que la fonction qui suit répond aux requêtes **POST** dont le chemin est `/predict`. Concrètement, FastAPI ajoute une entrée à sa table de routage : (**POST**, `/predict`) → `predict`. Quand une requête correspond, FastAPI lit le corps, le valide et appelle la fonction.

data: Features

L'annotation indique que `data` vient du corps JSON et doit respecter le schéma `Features`. Quand la fonction s'exécute, `data` est déjà un objet valide.

x = [...]

Scikit-learn attend un tableau à *deux dimensions* : une ligne par observation, une colonne par variable. Pour une seule observation, une liste de listes.

model.predict(x)

Renvoie un tableau NumPy, par exemple `array([1])`.

int(y[0])

Convertit le `numpy.int64` en entier Python. Sans cette conversion, la sérialisation JSON peut échouer.

9.4 Tester

```
$ uvicorn main:app --reload  
$ curl -X POST http://127.0.0.1:8000/predict \  
    -H "Content-Type: application/json" \  
    -d '{"age": 25, "income": 350000, "education": 3}' \  
{"prediction":1}
```

Ouvrez aussi `/docs` : l'endpoint **POST** `/predict` apparaît avec son schéma d'entrée, et vous pouvez l'essayer directement.

9.5 Les limites de cette première version

Cette version fonctionne mais reste fragile

- l'**ordre** des variables dans `x` doit être identique à l'entraînement, sans aucune vérification ;
- aucun **prétraitement** : un modèle entraîné avec un `StandardScaler` donnerait des résultats faux ;

- les **catégories** (texte) ne sont pas gérées ;
- aucune **gestion d'erreurs** si le fichier `model.joblib` est absent ;
- tout est dans un seul fichier : difficile à tester et à faire évoluer.

Les parties suivantes corrigent ces points un par un.

Question de compréhension 9.1

Pourquoi `model.predict({"age": 25})` échoue-t-il alors que `model.predict([[25, 350000, 3]])` fonctionne ?

Exercices

Exercice 6 • Niveau 4 — Charger un modèle .joblib hors API

Objectif : manipuler l'inférence sans réseau. **Consignes** : exécutez `train\_toy.py`, puis écrivez un script `check.py` qui charge `model.joblib`, prédit pour trois observations et affiche le type du résultat. **Résultat attendu** : trois prédictions et la mention `numpy.int64`.

Correction

Correction — Charger un modèle .joblib hors API

```
import joblib

model = joblib.load("model.joblib")
X_new = [[25, 350000, 3], [40, 120000, 1], [55, 500000, 5]]
preds = model.predict(X_new)
print(preds, type(preds[0]))  # [1 0 1] <class 'numpy.int64'>
```

Le type `numpy.int64` explique pourquoi on écrit `int(...)` avant de répondre en JSON.

PARTIE VI

Modèles `.pkl` et `.joblib`

Sauvegarder et charger un modèle

10.1 Pourquoi sauvegarder un modèle ?

Un modèle entraîné n'existe qu'en mémoire vive : si le programme s'arrête, il disparaît. Sauvegarder (*sérialiser*) un modèle écrit son état dans un fichier, pour le recharger plus tard sans réentraîner.

10.2 Que contient le fichier ?

Le fichier contient l'*objet Python* tel qu'il était en mémoire :

- les **hyperparamètres** choisis (`n_estimators=50`, `max_depth`, ...);
- les **paramètres appris** (arbres, coefficients, moyennes et écarts-types d'un `StandardScaler`, catégories d'un `OneHotEncoder`);
- la structure interne nécessaire à l'inférence.

Il ne contient pas le jeu de données (sauf cas particuliers, comme certains modèles à voisinage), ni votre notebook, ni les étapes de nettoyage faites ailleurs. Ce dernier point est crucial : *si le prétraitement n'est pas dans l'objet sauvegardé, il est perdu.*

10.3 pickle ou joblib ?

	pickle	joblib
Origine	Bibliothèque standard Python	Bibliothèque externe, proche de scikit-learn
Points forts	Universel	Efficace avec les grands tableaux NumPy
Extensions usuelles	<code>.pkl</code> , <code>.pickle</code>	<code>.joblib</code>

Pour des modèles Scikit-learn, joblib est généralement préféré.

```
import joblib

joblib.dump(model, "models/model.joblib")    # sauvegarde
model = joblib.load("models/model.joblib")    # chargement
```

Avec pickle :

```
import pickle

with open("model.pkl", "wb") as f:
```

```
pickle.dump(model, f)
with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

10.4 Deux dangers à connaître

1. Ne chargez jamais un fichier pickle/joblib de source inconnue

Ces formats peuvent exécuter du code arbitraire au chargement. Un fichier `.pkl` piégé peut compromettre votre serveur. Ne chargez que des fichiers que *vous* avez produits ou qui viennent d'une source de confiance.

2. La version des bibliothèques compte

Un modèle sauvegardé avec une version de Scikit-learn peut échouer, ou pire, se comporter différemment avec une autre version. Notez la version utilisée à l'entraînement et installez la même dans l'API (par exemple en la figeant dans `requirements.txt`).

10.5 Sauvegarder aussi les métadonnées

Un modèle seul ne dit pas quelles colonnes il attend, ni comment il a été produit. Une bonne pratique : enregistrer à côté un petit fichier JSON.

```
import json, sklearn

metadata = {
    "version": "1.0.0",
    "sklearn_version": sklearn.__version__,
    "features": ["age", "income", "education"],
    "metrics": {"f1": 0.91},
}

with open("models/metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)
```

L'API pourra lire ce fichier au démarrage, vérifier la compatibilité et renvoyer la version du modèle dans ses réponses : indispensable pour déboguer quand plusieurs versions coexistent.

À retenir

Sauvegarder, c'est *figer l'état* d'un objet. Tout ce qui est nécessaire pour prédire doit se trouver dans cet objet (ou dans un fichier associé), pas dans votre tête ni dans votre notebook.

Question de compréhension 10.1

Pourquoi sauvegarder un Pipeline complet plutôt que seulement le modèle final?

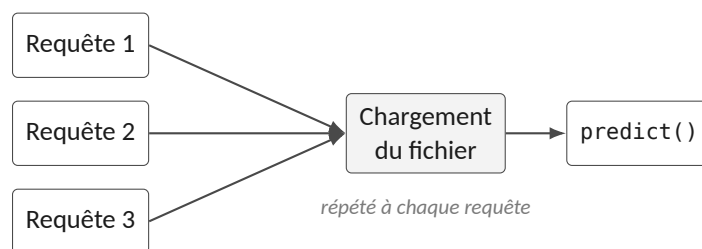
(Réfléchissez à ce que contient un `StandardScaler` ajusté.)

Quand et comment charger le modèle

11.1 La mauvaise idée : charger à chaque requête

```
@app.post("/predict")
def predict(data: Features):
    model = joblib.load("model.joblib")    # à NE PAS faire
    ...
```

À chaque requête, le serveur relirait le fichier sur le disque, le désérialiserait et reconstruirait l'objet en mémoire. Pour un petit modèle, ce coût est de quelques millisecondes ; pour un modèle volumineux (centaines de Mo), il peut atteindre plusieurs secondes. La latence devient imprévisible, le disque est sollicité inutilement et la mémoire est mise à rude épreuve quand plusieurs requêtes arrivent ensemble.



11.2 La bonne idée : charger une seule fois

Le modèle est *en lecture seule* pendant l'inférence : il peut donc être partagé entre toutes les requêtes. On le charge au démarrage, et on le garde en mémoire.

Option 1 : au niveau du module

```
model = joblib.load("model.joblib")    # exécuté une fois, à l'import du module
```

C'est la solution la plus simple (celle du chapitre précédent). Défaut : un fichier manquant fait échouer l'import, avec un message peu clair, et c'est difficile à remplacer dans les tests.

Option 2 : au démarrage de l'application (*lifespan*)

FastAPI permet d'exécuter du code au démarrage et à l'arrêt via une fonction *lifespan* :

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
```

```
import joblib

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = joblib.load("models/model.joblib")    # démarrage
    yield                                                    # l'API tourne ici
    # arrêt : libérer des ressources si nécessaire

app = FastAPI(lifespan=lifespan)

@app.post("/predict")
def predict(data: Features, request: Request):
    model = request.app.state.model                        # déjà en mémoire
    ...
```

- tout ce qui précède `yield` s'exécute *une fois*, avant d'accepter la première requête ;
- `app.state` est un espace de stockage rattaché à l'application, accessible depuis les routes via `request.app.state` ;
- si le chargement échoue, l'erreur est claire et le serveur ne démarre pas (ou démarre en mode dégradé, voir le projet final).

Option 3 : une dépendance

Pour garder les routes propres, on peut encapsuler l'accès au modèle dans une *dépendance* FastAPI (`Depends`), qui lit `app.state`. Cette technique, utilisée dans le projet final, facilite aussi les tests en permettant de remplacer le modèle par un faux.

À retenir

Charger *au démarrage*, utiliser *à chaque requête*. Le chargement est un coût d'installation ; l'inférence est un coût d'usage.

Plusieurs processus

Si vous lancez plusieurs « workers » (voir chapitre 26), chaque processus charge sa propre copie du modèle. La mémoire utilisée est multipliée d'autant : à prendre en compte pour les gros modèles.

Question de compréhension 11.1

Pourquoi peut-on partager un même objet modèle entre plusieurs requêtes simultanées sans risque, alors qu'on ne partagerait pas une variable globale modifiée à chaque requête ?

Exercices

Exercice 7 • Niveau 5 — Créer /predict

Objectif : exposer un modèle chargé au démarrage. **Consignes** : à partir de `model.joblib` (trois variables du chapitre 9), écrivez une API avec une fonction *lifespan*, un schéma Pydantic `Features` avec contraintes (`age` entre 18 et 100, `education` entre 1 et 5) et un endpoint `POST /predict` renvoyant `\{"prediction": ..., "probability": ...\\}`. **Résultat attendu** : réponse 200 pour une entrée valide, 422 pour `age = 5`.

Correction

Correction — Créer /predict

```
from contextlib import asynccontextmanager
import joblib
from fastapi import FastAPI, Request
from pydantic import BaseModel, Field

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = joblib.load("model.joblib")
    yield

app = FastAPI(lifespan=lifespan)

class Features(BaseModel):
    age: int = Field(..., ge=18, le=100)
    income: float = Field(..., ge=0)
    education: int = Field(..., ge=1, le=5)

@app.post("/predict")
def predict(data: Features, request: Request):
    model = request.app.state.model
    x = [[data.age, data.income, data.education]]
    return {
        "prediction": int(model.predict(x)[0]),
        "probability": float(model.predict_proba(x)[0][1]),
    }
```

PARTIE VII

Prétraitement et Pipelines

Le prétraitement fait partie du modèle

Le principe central de cette partie

Une API ML ne doit pas seulement *charger* le modèle : elle doit reproduire *exactement* la préparation des données utilisée pendant l'entraînement.

12.1 Le décalage entre entraînement et service

Pendant l'entraînement, vos données ont subi des transformations : mise à l'échelle, encodage des catégories, imputation, choix et ordre des colonnes. Le modèle a appris *dans cet espace transformé*. S'il reçoit en production des données brutes, ou transformées autrement, ses prédictions deviennent fausses, *sans erreur visible*. Ce phénomène est souvent appelé *training-serving skew*.

12.2 Les pièges classiques

Mise à l'échelle (StandardScaler, MinMaxScaler)

Un `StandardScaler` apprend, sur le jeu d'entraînement, une moyenne et un écart-type par colonne, puis calcule $z = (x - \mu) / \sigma$. Deux erreurs typiques :

- oublier de l'appliquer à l'inférence ;
- le *réajuster* (`fit`) sur la nouvelle donnée : avec une seule ligne, l'écart-type vaut zéro et les valeurs n'ont plus de sens. À l'inférence, on appelle seulement `transform`, avec les μ et σ mémorisés.

Encodage des catégories (OneHotEncoder)

Le modèle attend une colonne binaire par catégorie vue à l'entraînement, dans un ordre précis. À l'inférence, une catégorie inconnue (« doctorat » alors que l'encodeur a vu seulement « secondaire », « licence », « master ») lève une erreur, sauf si l'on a choisi `handle_unknown="ignore"`.

Ordre des variables et colonnes attendues

Avec des tableaux NumPy, *seul l'ordre compte* ; avec des `DataFrame` et un `ColumnTransformer`, ce sont les *noms* des colonnes. Dans les deux cas, une erreur d'ordre ou de nom fausse ou casse la prédiction.

Valeurs manquantes

Si l'entraînement utilisait un `SimpleImputer(strategy="median")`, l'API doit appliquer la même médiane (celle de l'entraînement), pas une valeur choisie à la main.

12.3 Démonstration chiffrée

Un petit modèle de régression logistique est entraîné sur `age` et `income` *mis à l'échelle*. Pour le même client (35 ans, revenu 4 000 000), on obtient :

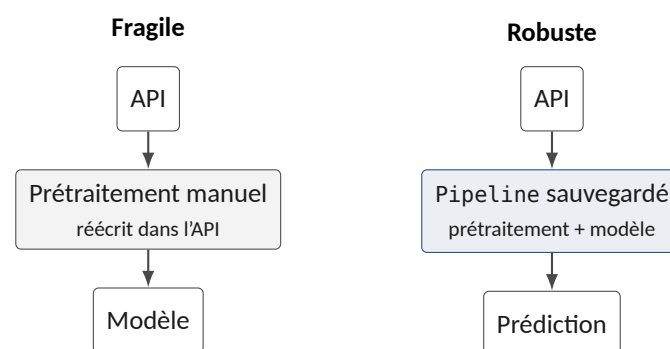
Façon de préparer la donnée	Probabilité d'accord
Avec le scaler d'entraînement (correct)	0,956
Sans mise à l'échelle	1,000
Avec le scaler mais colonnes inversées	0,000

Les trois appels réussissent, aucune exception, trois résultats différents. C'est exactement ce qui rend ce bug dangereux : il est *silencieux*.

Attention

Écrire `model.predict(data)` avec des données *brutes* sur un modèle entraîné avec des données *transformées* est l'une des erreurs les plus graves d'un déploiement ML, précisément parce que rien ne plante.

12.4 Deux architectures



Dans l'architecture *fragile*, le prétraitement est dupliqué : une fois dans le notebook, une fois dans l'API. Il suffit qu'une des deux copies évolue pour que les deux divergent. Dans l'architecture *robuste*, le prétraitement et le modèle forment *un seul objet*, entraîné, sauvegardé et rechargé ensemble.

Question de compréhension 12.1

Pourquoi le `StandardScaler` ajusté doit-il être conservé avec le modèle, et pas recalculé à l'inférence ?

Un Pipeline complet, de l'entraînement à l'API

13.1 Construire le Pipeline

train_pipeline.py

```
import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

NUM = ["age", "income"]
CAT = ["education"]

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), NUM),
    ("cat", OneHotEncoder(handle_unknown="ignore"), CAT),
])

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=1000)),
])

pipeline.fit(X_train, y_train)          # X_train : un DataFrame avec les 3 colonnes
joblib.dump(pipeline, "models/model.joblib")
```

ColumnTransformer

Applique des transformations différentes selon les colonnes : ici, imputation puis standardisation pour les numériques, encodage *one-hot* pour les catégories.

Pipeline

Enchaîne le prétraitement et le modèle. `fit` ajuste chaque étape sur le jeu d'entraînement ; `predict` applique *toutes* les transformations puis le modèle.

handle_unknown="ignore"

Une catégorie jamais vue produit un vecteur de zéros au lieu d'une exception. C'est un choix

de conception : tolérant, mais il masque un problème de données. Nous verrons en « Pydantic + Literal » comment refuser plutôt les catégories inconnues dès l'entrée de l'API.

13.2 Pourquoi c'est idéal pour une API

- **Un seul fichier** à déployer, un seul `joblib.load`.
- **Aucune duplication** du prétraitement dans l'API.
- **Cohérence garantie** : les paramètres appris (moyennes, écarts-types, catégories, médianes) sont ceux de l'entraînement.
- **Entrée brute** : l'API envoie un `DataFrame` avec les colonnes d'origine ; le Pipeline fait le reste.
- **Évolution simple** : changer de modèle ou de prétraitement revient à ré-exporter le Pipeline.

13.3 Recharger le Pipeline dans FastAPI

```
import pandas as pd

def to_dataframe(data) -> pd.DataFrame:
    # colonnes dans l'ordre et avec les noms de l'entraînement
    return pd.DataFrame([data.model_dump()], columns=["age", "income", "education"])

@app.post("/predict")
def predict(data: Features, request: Request):
    pipeline = request.app.state.model
    df = to_dataframe(data)
    proba = float(pipeline.predict_proba(df)[0, 1])
    return {"prediction": int(proba >= 0.5), "probability": round(proba, 4)}
```

- `data.model_dump()` convertit l'objet Pydantic en dictionnaire ;
- `columns=[...]` impose les noms et l'ordre des colonnes : on reconstruit la structure du jeu d'entraînement ;
- `predict_proba` donne la probabilité de la classe positive, souvent plus utile qu'un 0 ou 1.

13.4 Ce qui peut encore mal se passer

Les tests faits sur un Pipeline entraîné pour ce chapitre donnent :

Entrée	Comportement
Colonne <code>education</code> absente du <code>DataFrame</code>	<code>ValueError: columns are missing</code>
<code>age</code> manquant (<code>NaN</code>)	Médiane d'entraînement utilisée
Catégorie inconnue « <code>doctorat</code> »	Vecteur de zéros (avec <code>ignore</code>)

La validation Pydantic (champs obligatoires, `Literal` pour les catégories) arrête la plupart de

ces cas *avant* le Pipeline. C'est la défense en deux couches : **Pydantic protège l'entrée, le Pipeline garantit la transformation.**

À retenir

Sauvegardez le **Pipeline**, pas seulement le modèle. L'API envoie des données brutes dans le bon format ; l'objet sauvegardé sait comment les préparer.

Question de compréhension 13.1

Pourquoi sauvegarder un Pipeline plutôt que seulement le modèle ? Donnez deux risques évités.

Exercices

Exercice 8 • Niveau 6 — Ajouter un Pipeline Scikit-learn

Objectif : intégrer le prétraitement au modèle. **Consignes** : (a) générez un petit DataFrame avec **age**, **income** (numériques) et **education** (catégorielle, avec valeurs manquantes sur **age**) ; (b) construisez un **Pipeline** avec imputation, standardisation, encodage *one-hot* et **LogisticRegression** ; (c) sauvegardez-le ; (d) faites un script qui le recharge et prédit pour une ligne contenant une catégorie inconnue. **Résultat attendu** : une prédiction sans exception.

Correction

Correction — Ajouter un Pipeline Scikit-learn

```
import joblib, numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

rng = np.random.default_rng(1)
n = 300
df = pd.DataFrame({
    "age": rng.integers(18, 70, n).astype(float),
    "income": rng.normal(3e6, 1e6, n),
    "education": rng.choice(["secondaire", "licence", "master"], n),
})
df.loc[:20, "age"] = np.nan
y = (df["income"] > 3e6).astype(int)

pre = ColumnTransformer([
```

```
(("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                    ("sc", StandardScaler())])), ["age", "income"]),
("cat", OneHotEncoder(handle_unknown="ignore"), ["education"]),
])
pipe = Pipeline([("preprocessor", pre), ("model", LogisticRegression())]).fit(df,
→y)
joblib.dump(pipe, "pipe.joblib")

pipe2 = joblib.load("pipe.joblib")
new = pd.DataFrame({"age": [40], "income": [3.5e6], "education": ["doctorat"]})
print(pipe2.predict_proba(new)[0, 1])
```

PARTIE VIII

Architecture d'une API ML

D'un seul fichier à une architecture propre

Un projet qui commence dans un fichier `main.py` de 40 lignes devient vite illisible quand on y ajoute validation, modèle, erreurs, configuration, tests. L'architecture sert à répondre à une question : *où mettre quoi, et pourquoi ?* Nous la faisons évoluer par étapes, chacune justifiée par un problème précis.

14.1 Étape 1 : tout dans `main.py`

```
ml_api/|—
  main.py|—
  model.joblib|—
  requirements.txt
```

Parfait pour apprendre ou prototyper. *Problème* : dès que le fichier dépasse une centaine de lignes, on ne distingue plus l'API, la validation et le modèle.

14.2 Étape 2 : séparer les responsabilités

```
ml_api/|—
  app/|
    |— main.py          # création de l'app et des routes|
    |— schemas.py       # modèles Pydantic (entrées / sorties)|
    |— model.py          # chargement et appel du modèle|
    |— preprocessing.py  # préparation des données (si nécessaire)|—
  models/|
    |— model.joblib|—
  requirements.txt|—
  README.md
```

Chaque fichier a *une raison de changer* :

- `schemas.py` change quand le *contrat* avec les clients change ;
- `model.py` change quand le *modèle* change ;
- `main.py` change quand on ajoute une *route* ;
- `preprocessing.py` change si la préparation évolue (avec un Pipeline, il disparaît souvent).

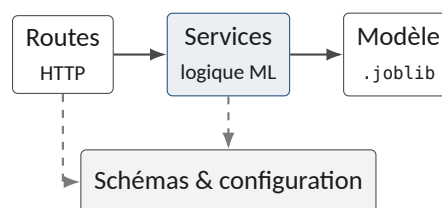
Pourquoi séparer ?

Pour pouvoir modifier une partie sans casser les autres, tester chaque partie isolément, et permettre à plusieurs personnes de travailler sans se marcher dessus.

14.3 Étape 3 : routes, services, configuration

Quand l'API compte plusieurs endpoints et modèles, on regroupe par rôle :

```
project/|—
  app/|
    |— main.py                # assemblage de l'application|
    |— routes/|
    |   |— prediction.py      # endpoints HTTP|
    |— schemas/|
    |   |— prediction.py      # contrats d'entrée / sortie|
    |— services/|
    |   |— model_service.py   # logique du modèle (aucun HTTP ici)|
    |— core/|
    |   |— config.py          # configuration, variables d'environnement|—
  models/|
    |— model.joblib|—
  tests/|—
  requirements.txt|—
  README.md
```



La règle essentielle : les **routes** parlent HTTP (lire une requête, choisir un code de statut), les **services** parlent ML (préparer les données, appeler **predict**). Un service ne connaît ni **Request** ni **HTTPException**. Cela permet de tester le service sans lancer de serveur.

Ne sur-architectez pas

Une architecture complexe sur un projet de 50 lignes est une perte de temps. Ajoutez un niveau de structure *quand le besoin apparaît*, pas avant.

14.4 Un exemple de route fine

routes/prediction.py

```
from fastapi import APIRouter, Depends, Request
from app.schemas.prediction import Features, PredictionResponse
from app.services.model_service import ModelService

router = APIRouter(prefix="/api/v1", tags=["prédiction"])

def get_service(request: Request) -> ModelService:
    return request.app.state.model_service
```

```
@router.post("/predict", response_model=PredictionResponse)
def predict(data: Features, service: ModelService = Depends(get_service)):
    return service.predict(data)      # la route ne fait que déléguer
```

- **APIRouter** regroupe des routes dans un module et les rattache à l'application avec `app.include_router(r)`
- **Depends** fournit la dépendance (ici le service) à la route et permet de la remplacer dans les tests ;
- la route reste *courte* : elle reçoit, délègue, renvoie.

Question de compréhension 14.1

Dans quel fichier placeriez-vous : (a) le calcul de `predict_proba` ; (b) la contrainte « age entre 18 et 100 » ; (c) la lecture de la variable d'environnement `API_KEY` ?

Faire évoluer l'API : versions, plusieurs modèles, base de données

15.1 Versionner une API

Une fois qu'une application mobile utilise votre API, vous ne contrôlez plus ses clients. Si vous renommez un champ (`income` devient `annual\_income`), toutes les applications existantes cessent de fonctionner. La parade : **versionner**.

```
POST /api/v1/predict      # contrat historique, reste disponible
POST /api/v2/predict      # nouveau contrat
```

- un changement **compatible** (ajouter un champ optionnel, ajouter un endpoint) n'impose pas de nouvelle version ;
- un changement **cassant** (renommer ou supprimer un champ, changer un type, modifier la signification d'une valeur) impose **v2** ;
- on garde **v1** en service le temps que les clients migrent, puis on le retire après un préavis annoncé.

En FastAPI, il suffit de créer un `APIRouter` par version : `APIRouter(prefix="/api/v1")` et `APIRouter(prefix="/api/v2")`.

15.2 Plusieurs modèles : une API, plusieurs endpoints, plusieurs services

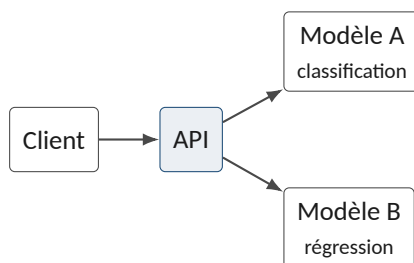
```
POST /api/v1/model/classification
POST /api/v1/model/regression
POST /api/v1/model/risk
```

Trois organisations possibles :

Choix	Quand	Limite
Une API, un endpoint par modèle	Modèles liés, même équipe, mêmes entrées	Tout se déploie ensemble
Une API, endpoints et services séparés	Les modèles ont des cycles de vie différents mais restent dans le même processus	Mémoire partagée
Plusieurs services indépendants	Équipes distinctes, charges très différentes, déploiements indépendants	Complexité d'exploitation (introduction aux microservices)

Commencez simple : une seule API et un endpoint par modèle. Séparez en services quand un besoin réel (charge, équipe, cadence de mise à jour) le justifie.

15.3 Deux modèles dans une même API



Le service charge *les deux* pipelines au démarrage, et chaque endpoint appelle le bon :

```

class ModelService:
    def load(self):
        self.classifier = joblib.load("models/approval_pipeline.joblib")
        self.regressor = joblib.load("models/amount_pipeline.joblib")

    def predict_approval(self, df): ...
    def predict_amount(self, df): ...

@router.post("/predict")          # modèle A
def predict(...): return service.predict_approval(...)

@router.post("/predict/amount")   # modèle B
def predict_amount(...): return service.predict_amount(...)
  
```

Le projet final (chapitre 27) implémente exactement ce schéma : un classifieur d'octroi de prêt et un régresseur de montant maximal, derrière la même API.

15.4 API et base de données

Jusqu'ici l'API est *sans mémoire*. Dans un produit réel, elle s'appuie souvent sur une base de données :



- **Lire des données** : enrichir la requête avec des informations connues (historique d'un client) avant la prédiction. Le client n'envoie alors qu'un identifiant.
- **Enregistrer une prédiction** : garder une trace de l'entrée, de la sortie et de la version du modèle, pour l'audit et le suivi de qualité.
- **Récupérer l'historique** : GET `/predictions?client\_id=12`.
- **Stocker les résultats** pour comparer plus tard prédiction et réalité (surveillance de dérive).

Attention

Un appel à la base de données ajoute de la latence et un point de panne. Ne l'introduisez que si le besoin existe, et séparez-le dans une couche dédiée pour ne pas mélanger logique ML et accès aux données.

Question de compréhension 15.1

Une équipe veut renommer le champ `income` en `annual\_income`. Quelles sont les conséquences si elle modifie `/api/v1/predict` directement ? Que faut-il faire à la place ?

Exercices

Exercice 9 • Niveau 9 — Créer une architecture propre

Objectif : structurer un projet. **Consignes** : à partir de l'API du chapitre 11, créez l'arborescence `app/` avec `main.py`, `schemas.py`, `model\_service.py` et `routes.py`. Les routes ne doivent contenir aucune logique ML. **Résultat attendu** : l'API répond comme avant ; `uvicorn app.main:app` démarre sans erreur.

Correction

Correction — Créer une architecture propre

```
# app/schemas.py
from pydantic import BaseModel, Field
class Features(BaseModel):
    age: int = Field(..., ge=18, le=100)
    income: float = Field(..., ge=0)
    education: int = Field(..., ge=1, le=5)

# app/model_service.py
import joblib
class ModelService:
    def __init__(self, path): self.path, self.model = path, None
    def load(self): self.model = joblib.load(self.path)
    def predict(self, d):
        x = [[d.age, d.income, d.education]]
        return {"prediction": int(self.model.predict(x)[0])}

# app/routes.py
from fastapi import APIRouter, Depends, Request
from app.schemas import Features
router = APIRouter(prefix="/api/v1")
def get_service(request: Request): return request.app.state.service
@router.post("/predict")
```

```
def predict(data: Features, service=Depends(get_service)):
    return service.predict(data)
```

```
# app/main.py
```

```
from contextlib import asynccontextmanager
```

```
from fastapi import FastAPI
```

```
from app.model_service import ModelService
```

```
from app.routes import router
```

```
@asynccontextmanager
```

```
async def lifespan(app):
```

```
    app.state.service = ModelService("models/model.joblib")
```

```
    app.state.service.load()
```

```
    yield
```

```
app = FastAPI(lifespan=lifespan)
```

```
app.include_router(router)
```

PARTIE IX

Tests, erreurs et documentation

Gérer les erreurs proprement

Une API en service reçoit des données imprévues, perd parfois son modèle, tombe sur des cas limites. Une bonne API ne « plante » pas : elle *répond*, avec un code de statut précis et un message exploitable.

16.1 Les codes de statut à connaître

Code	Nom	Quand l'utiliser dans une API ML
200	OK	Prédiction réussie
400	Bad Request	Requête incorrecte non couverte par la validation (règle métier violée)
401	Unauthorized	Authentification absente (clé d'API manquante)
403	Forbidden	Authentifié mais non autorisé (clé invalide)
404	Not Found	Ressource ou modèle demandé inexistant
422	Unprocessable Entity	Corps bien formé mais données invalides (validation Pydantic)
500	Internal Server Error	Erreur inattendue côté serveur (bug, erreur dans <code>predict()</code>)
503	Service Unavailable	Service temporairement inutilisable (modèle non chargé)

Règle simple : 4xx signifie « le client doit corriger sa demande », 5xx signifie « le problème est de mon côté ».

16.2 Les erreurs de validation (422)

Elles sont produites automatiquement par FastAPI et Pydantic (chapitre 7). Le format par défaut est riche mais verbeux ; on peut le simplifier avec un gestionnaire d'exception :

```
from fastapi import Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    errors = [
        {"field": ".".join(str(p) for p in e["loc"][1:]), "message": e["msg"]}
        for e in exc.errors()
    ]
    return JSONResponse(status_code=422,
                        content={"detail": "Données invalides", "errors": errors})
```

Pour `\{"age": 3\}` la réponse devient :

```
{"detail": "Données invalides",
  "errors": [{"field": "age", "message": "Input should be greater than or equal to 18"}]}
```

16.3 Lever une erreur volontairement : HTTPException

```
from fastapi import HTTPException

@app.get("/models/{name}")
def get_model(name: str):
    if name != "risk":
        raise HTTPException(status_code=404, detail=f"Modèle '{name}' introuvable")
    return {"name": name}
```

`raise HTTPException` interrompt la fonction et renvoie immédiatement la réponse avec le code et le message demandés.

16.4 Les erreurs du modèle

Une erreur dans `predict()` (colonne manquante, version incompatible, valeur inattendue) ne doit pas fuiter vers le client sous forme de *stack trace*. On l'attrape dans la couche service, on la *journalise* avec tous les détails, puis on lève une erreur métier dédiée :

```
class PredictionError(RuntimeError):
    """Erreur survenue pendant predict()."""

    try:
        classes = self.classifier.predict(df)
    except Exception as exc:
        logger.exception("Échec de predict()")          # détails dans les logs
        raise PredictionError("Erreur interne pendant la prédiction") from exc

@app.exception_handler(PredictionError)
async def prediction_error_handler(request: Request, exc: PredictionError):
    return JSONResponse(status_code=500, content={"detail": str(exc)})
```

Ne jamais exposer les détails internes

Un message d'erreur qui révèle des chemins de fichiers, des noms de colonnes internes ou une trace Python aide un attaquant. Le client reçoit un message générique ; les détails vont dans les journaux.

À retenir

Une bonne réponse d'erreur répond à trois questions : *que s'est-il passé* (code), *pourquoi*

(message) et *que faire* (champ à corriger, réessayer plus tard).

Question de compréhension 16.1

Un client envoie une valeur d'âge négative. Quel code attendez-vous ? Et si le fichier `model.joblib` est introuvable au démarrage ?

Tester une API à la main

Avant d'écrire des tests automatiques, il faut savoir envoyer des requêtes de plusieurs façons.

Outil	Usage	Limite
Swagger UI (/docs)	Explorer et essayer l'API	Peu adapté aux scénarios répétés
Navigateur	Tester les GET	Ne sait pas envoyer de corps POST
Postman	Collections de requêtes sauvegardées	Application à installer
curl	Scripts, terminal, intégration continue	Syntaxe parfois lourde
Python requests	Scénarios programmables	Demande d'écrire du code
Tests automatisés	Vérification systématique	Se met en place une fois

17.1 curl

```
$ curl -X POST http://127.0.0.1:8000/api/v1/predict \
  -H "Content-Type: application/json" \
  -H "X-API-Key: dev-key" \
  -d '{"age": 35, "income": 4200000, "education": "licence",
    "employment_type": "salarie", "debt_ratio": 0.28}'
```

- -X POST : la méthode; -H : un en-tête; -d : le corps;
- Content-Type: application/json dit au serveur comment lire le corps;
- l'option -i affiche aussi le code de statut et les en-têtes.

17.2 Python avec requests

```
import requests

payload = {"age": 35, "income": 4_200_000, "education": "licence",
           "employment_type": "salarie", "debt_ratio": 0.28}

response = requests.post(
    "http://127.0.0.1:8000/api/v1/predict",
    json=payload,
    headers={"X-API-Key": "dev-key"},
    timeout=5,
)
print(response.status_code)      # 200
print(response.json())           # {'prediction': 1, 'label': 'accordé', ...}
```

Ce qui se passe réellement : `json=payload` *séréalise* le dictionnaire en texte JSON et ajoute automatiquement l'en-tête `Content-Type: application/json`; `requests` ouvre une connexion TCP, envoie la requête HTTP, attend la réponse (au plus 5 secondes); `response.json()` *déséréalise* le corps en dictionnaire Python.

Pensez au timeout

Sans `timeout`, un client peut attendre indéfiniment un serveur bloqué. Définissez-le toujours.

Question de compréhension 17.1

Pourquoi le navigateur ne suffit-il pas pour tester `POST /predict` ?

Tests automatisés avec pytest

Un test automatique rejoue vos scénarios à chaque modification : il détecte les régressions avant vos utilisateurs. Que teste-t-on dans une API de prédiction ? *Le contrat* (formats, codes, validations), pas la qualité statistique du modèle (qui se mesure à l'entraînement).

18.1 Mise en place

```
$ pip install pytest httpx
$ pytest -q
```

FastAPI fournit un `TestClient` qui appelle l'application *sans lancer de serveur* :

tests/conftest.py

```
import pytest
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture(scope="session")
def client():
    with TestClient(app) as c:      # "with" déclenche le lifespan : le modèle est chargé
        yield c

@pytest.fixture
def headers():
    return {"X-API-Key": "dev-key"}

@pytest.fixture
def valid_payload():
    return {"age": 35, "income": 4200000, "education": "licence",
            "employment_type": "salarie", "years_employed": 6.5, "debt_ratio": 0.28}
```

Une *fixture* pytest prépare un objet réutilisable par les tests qui le demandent en argument.

18.2 Les quatre types de tests essentiels

tests/test_prediction.py

```
def test_home(client):                                # 1. endpoint
    r = client.get("/")
    assert r.status_code == 200
```

```
def test_predict_ok(client, headers, valid_payload):    # 2. prédiction
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 200
    body = r.json()
    assert body["prediction"] in (0, 1)
    assert 0.0 <= body["probability"] <= 1.0

def test_validation_error_422(client, headers, valid_payload):    # 3. validation
    valid_payload["age"] = 5
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 422

def test_missing_api_key_401(client, valid_payload):    # 4. erreur d'accès
    assert client.post("/api/v1/predict", json=valid_payload).status_code == 401
```

Lancez `pytest -q` : chaque fonction préfixée par `test\_` est exécutée, et `assert` déclenche un échec si la condition est fausse.

18.3 Remplacer le modèle par un faux

Pour tester la route sans dépendre d'un vrai fichier modèle, on remplace la dépendance :

```
class FakeService:
    def predict_approval(self, items):
        return [{"prediction": 1, "label": "accordé", "probability": 0.9,
                  "model_version": "test"}]

app.dependency_overrides[get_service] = lambda: FakeService()
```

Les tests deviennent rapides et déterministes, car ils ne vérifient que la *logique de l'API*.

À retenir

Un test d'API vérifie ce qu'un client verrait : code de statut, structure de la réponse, comportement face à de mauvaises données. Si ces tests passent, le contrat est respecté.

Question de compréhension 18.1

Pourquoi écrit-on `with TestClient(app) as c` plutôt que `c = TestClient(app)` quand le modèle est chargé dans une fonction *lifespan* ?

Documentation automatique

19.1 OpenAPI, Swagger UI, ReDoc

- **OpenAPI** est une *norme* de description d'API, au format JSON. FastAPI la génère automatiquement à partir de votre code et la publie sur `/openapi.json`.
- **Swagger UI** (`/docs`) lit cette description et propose une interface interactive.
- **ReDoc** (`/redoc`) offre une présentation plus orientée lecture.

19.2 Pourquoi la documentation est importante

Une équipe frontend ne lit pas votre code Python. Elle consulte la documentation pour savoir quels champs envoyer, quelles réponses attendre et quelles erreurs gérer. Comme elle est *générée depuis le code*, elle ne devient jamais obsolète. Les équipes peuvent même générer automatiquement un client dans leur langage à partir de `/openapi.json`.

19.3 Documenter correctement un endpoint de prédiction

```
class LoanRequest(BaseModel):
    age: int = Field(..., ge=18, le=100, description="Âge en années")
    income: float = Field(..., ge=0, description="Revenu annuel")
    debt_ratio: float = Field(..., ge=0, le=1, description="Taux d'endettement (0 à 1)")

    model_config = ConfigDict(json_schema_extra={
        "example": {"age": 35, "income": 4200000, "education": "licence",
                    "employment_type": "salarie", "years_employed": 6.5,
                    "debt_ratio": 0.28}})

@router.post(
    "/predict",
    response_model=ApprovalResponse,
    summary="Décision d'octroi de prêt",
    responses={401: {"description": "Clé d'API manquante"},
              422: {"description": "Données invalides"}},
)
def predict(payload: LoanRequest, ...):
    """Reçoit les données d'un client et renvoie la décision du modèle
    de classification, avec la probabilité associée."""
```

- `description=` documente chaque champ ;
- l'exemple pré-remplit Swagger : un frontend peut essayer l'API en un clic ;

- `summary` et la *docstring* décrivent l'endpoint ;
- `responses` documente les erreurs possibles : c'est une partie du contrat.

Question de compréhension 19.1

Comment un développeur frontend peut-il comprendre votre API sans lire votre code ? Citez deux éléments précis de la documentation générée.

Exercices

Exercice 10 • Niveau 7 — Gérer les erreurs

Objectif : produire des réponses d'erreur propres. **Consignes** : à partir de l'API `/predict`, ajoutez (a) un gestionnaire de `RequestValidationError` qui renvoie la liste `errors` simplifiée, (b) une route `GET /models/{name\}` qui renvoie `404` si le nom n'est pas "risk". **Résultat attendu** : un `422` lisible et un `404` avec un message clair.

Exercice 11 • Niveau 8 — Tester l'API

Objectif : écrire des tests automatiques. **Consignes** : écrivez avec `pytest` quatre tests : succès, validation (`422`), route inconnue (`404`), clé d'API manquante (`401`). **Résultat attendu** : 4 passed.

Correction

Correction — Gérer les erreurs

Voir les extraits des sections « Les erreurs de validation » et « Lever une erreur volontairement » de ce chapitre : le gestionnaire reformate `exc.errors()` et la route lève `HTTPException(404, ...)`. Testés : `POST /predict` avec `age=3` renvoie `422` avec `errors[0].field == "age"` ; `GET /models/xyz` renvoie `404`.

Correction — Tester l'API

```
def test_ok(client, headers, valid_payload):
    assert client.post("/api/v1/predict", json=valid_payload, headers=headers).
        →status_code == 200

def test_422(client, headers, valid_payload):
    valid_payload["age"] = 5
    assert client.post("/api/v1/predict", json=valid_payload, headers=headers).
        →status_code == 422

def test_404(client):
    assert client.get("/route-inconnue").status_code == 404
```

```
def test_401(client, valid_payload):  
    assert client.post("/api/v1/predict", json=valid_payload).status_code == 401
```

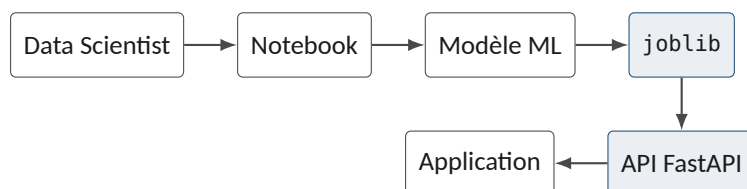
PARTIE X

API et applications clientes

Comment les Data Scientists utilisent réellement les API

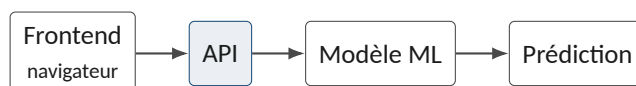
Une API n'est pas un exercice académique : c'est l'interface par laquelle votre travail rejoint le reste d'une organisation. Voici quatre scénarios réels.

20.1 Scénario 1 : du notebook à l'API



Le Data Scientist explore et entraîne dans un notebook, exporte le Pipeline en `joblib`, puis (souvent avec un ML engineer) enveloppe ce fichier dans une API. L'application ne voit que l'API.

20.2 Scénario 2 : une application web utilise le modèle



Un formulaire web collecte les informations d'un utilisateur, le code JavaScript envoie un `POST` à l'API et affiche le résultat. Le navigateur n'exécute *aucun* code Python.

Exemple JavaScript (côté frontend)

```
const response = await fetch("https://api.example.com/api/v1/predict", {
  method: "POST",
  headers: {"Content-Type": "application/json", "X-API-Key": key},
  body: JSON.stringify({age: 35, income: 4200000, education: "licence",
    employment_type: "salarie", debt_ratio: 0.28}),
});
const result = await response.json();
```

20.3 Scénario 3 : une application mobile appelle l'API

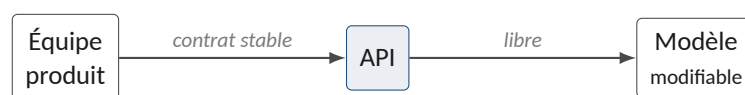
Une application Android ou iOS fonctionne exactement de la même façon : elle construit une requête HTTP avec la bibliothèque réseau de son langage (Kotlin, Swift) et lit le JSON de la réponse. Pour l'API, un client mobile, un site web ou un script Python sont *indiscernables* : tous envoient du HTTP.

Pourquoi un modèle Python sert-il une application mobile ?

Parce que la communication passe par HTTP et JSON, deux standards indépendants du langage. Le modèle reste en Python sur le serveur ; l'application mobile n'a besoin que d'une URL et d'un format de données.

20.4 Scénario 4 : une autre équipe utilise l'API sans connaître le modèle

L'équipe produit intègre votre `/predict` en se basant uniquement sur la documentation `/docs`. Elle ne sait pas si vous utilisez une régression logistique ou une forêt aléatoire, et n'a pas besoin de le savoir. Vous pouvez améliorer le modèle, changer d'algorithme ou ré-entraîner chaque semaine : tant que le *contrat* reste le même, rien ne casse de leur côté.



Question de compréhension 20.1

Pourquoi l'API crée-t-elle une séparation entre le modèle et l'application ? Quel avantage concret pour l'équipe qui ré-entraîne le modèle ?

L'API comme contrat

Contrat d'API

Une API est un **contrat** entre deux systèmes : elle promet que, si le client envoie des données dans un format précis, il recevra une réponse dans un format précis. Chacun peut évoluer librement tant qu'il respecte le contrat.

21.1 Ce que le contrat précise

Élément	Exemple dans notre API
Endpoint et méthode	POST /api/v1/predict
Format des données	JSON, Content-Type: application/json
Champs obligatoires	age, income, education, employment_type, debt_ratio
Champs optionnels	years_employed
Types et bornes	age entier entre 18 et 100 ; catégories restreintes
Format de la réponse	prediction, label, probability, model_version
Erreurs	401 clé absente, 403 clé invalide, 422 données invalides, 503 modèle indisponible
Documentation	/docs, /openapi.json
Stabilité	Les changements cassants passent par une nouvelle version (/v2)

21.2 Ce qui reste libre

Tout ce qui est *derrière* le contrat : l'algorithme, les hyperparamètres, la version de Scikit-learn, la façon de charger le modèle, l'organisation du code. C'est pourquoi l'équipe frontend peut utiliser votre modèle sans connaître un seul fichier Python.

Changement compatible et changement cassant

Passer d'une forêt aléatoire à un autre algorithme, sans changer les champs ni les types : *compatible*, personne ne le remarque. Renommer `income` en `annual\_income` : *cassant*, il faut une nouvelle version d'API.

À retenir

Concevoir une API, c'est d'abord concevoir un contrat *stable, explicite et documenté*. L'implémentation n'est qu'un détail interne qui peut changer.

Question de compréhension 21.1

Parmi ces changements, lesquels exigent une nouvelle version de l'API : (a) ajouter un champ optionnel en entrée; (b) renommer **probability** en **score**; (c) remplacer l'algorithme; (d) ajouter un champ `model\_version` dans la réponse?

Un client Python pour consommer l'API

À retenir

FastAPI n'est *pas* l'application finale. Il fournit un *service* que d'autres applications consomment. Le client est cette autre application.

22.1 Le code

client/client.py

```
import os
import sys
import requests

BASE_URL = os.getenv("API_URL", "http://127.0.0.1:8000")
API_KEY = os.getenv("API_KEY", "dev-key")
HEADERS = {"X-API-Key": API_KEY}

customer = {"age": 35, "income": 4_200_000, "education": "licence",
            "employment_type": "salarie", "years_employed": 6.5, "debt_ratio": 0.28}

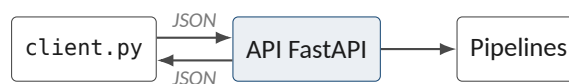
def main() -> None:
    try:
        r = requests.post(f"{BASE_URL}/api/v1/predict", json=customer,
                          headers=HEADERS, timeout=5)
    except requests.exceptions.ConnectionError:
        sys.exit("Impossible de joindre l'API : le serveur est-il lancé ?")

    if r.status_code == 200:
        data = r.json()
        print(f"Décision : {data['label']} (probabilité = {data['probability']:.2%})")
    elif r.status_code == 422:
        print("Données invalides :", r.json()["detail"])
    else:
        print(f"Erreur {r.status_code} :", r.json().get("detail"))

if __name__ == "__main__":
    main()
```

22.2 Le rôle de ce client

- il *ne contient aucun modèle* et n'importe pas Scikit-learn ;
- il construit le JSON conformément au contrat ;
- il *interprète* les codes de statut : succès, données invalides, autre erreur ;
- il gère le cas où le serveur est injoignable (`ConnectionError`) ;
- il lit l'URL et la clé dans l'*environnement* (`os.getenv`), pas dans le code.



Question de compréhension 22.1

Si le serveur n'est pas lancé, quelle exception Python lève `requests.post` ? Comment le client doit-il réagir ?

Exercices

Exercice 12 • Niveau 5 — Écrire un client

Objectif : consommer votre propre API. **Consignes** : écrivez un script qui lit un fichier CSV de clients, envoie chaque ligne à `POST /api/v1/predict`, et affiche le nombre de décisions « accordé ». Gérez les erreurs 422. **Résultat attendu** : un résumé, sans planter sur une ligne invalide.

Correction

Correction — Écrire un client

```

import csv, requests

URL, HEADERS = "http://127.0.0.1:8000/api/v1/predict", {"X-API-Key": "dev-key"}
accepted = invalid = 0

with open("clients.csv", newline="") as f:
    for row in csv.DictReader(f):
        payload = {"age": int(row["age"]), "income": float(row["income"]),
                  "education": row["education"],
                  "employment_type": row["employment_type"],
                  "debt_ratio": float(row["debt_ratio"])}
        r = requests.post(URL, json=payload, headers=HEADERS, timeout=5)
        if r.status_code == 200 and r.json()["prediction"] == 1:
            accepted += 1
        elif r.status_code == 422:

```

```
invalid += 1  
  
print(f"{accepted} accordés, {invalid} lignes invalides")
```

Pour de gros volumes, on préférera l'endpoint `/predict/batch` (chapitre 25).

PARTIE XI

Sécurité et bonnes pratiques

Les bases de la sécurité d'une API

Cette partie présente les *concepts*. Certaines notions avancées (OAuth, gestion fine des rôles) seront étudiées plus tard.

23.1 API publique ou privée

Une API **publique** est accessible à tout internaute ; une API **privée** est réservée à des clients identifiés (équipes internes, partenaires). Une API de prédiction contient souvent des données personnelles ou un modèle de valeur : elle est rarement publique sans contrôle.

23.2 Authentification et autorisation

- **Authentification** : *qui êtes-vous ?* (clé d'API, identifiant et mot de passe, jeton).
- **Autorisation** : *avez-vous le droit de faire cela ?* (rôles, permissions).

Les codes HTTP correspondent : 401 (non authentifié) et 403 (authentifié mais interdit).

23.3 Clés d'API

Une clé d'API est un secret que le client envoie dans un en-tête (ici X-API-Key). C'est le mécanisme le plus simple.

app/security.py

```
import secrets
from fastapi import Depends, HTTPException, status
from fastapi.security import APIKeyHeader
from app.config import Settings, get_settings

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def verify_api_key(key: str | None = Depends(api_key_header),
                  settings: Settings = Depends(get_settings)) -> None:
    if key is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Clé d'API manquante")
    if not secrets.compare_digest(key, settings.api_key):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Clé d'API invalide")
```

- APIKeyHeader lit l'en-tête et ajoute un bouton « Authorize » dans Swagger ;
- Depends fait de cette vérification une *dépendance* appliquée à toutes les routes du routeur ;

- `secrets.compare\_digest` compare en temps constant, ce qui évite certaines attaques par mesure du temps de réponse ;
- la valeur attendue vient de la configuration, *jamaïs* du code source.

23.4 JWT (JSON Web Token)

Un JWT est un jeton signé que le serveur remet après une authentification (par exemple identifiant et mot de passe). Il contient des informations (identité, rôle, date d'expiration) et une *signature* qui prouve qu'il n'a pas été modifié. Le client le joint à chaque requête dans `Authorization: Bearer <jeton>`. Le serveur n'a pas besoin de mémoriser les sessions (cohérent avec le principe *stateless* de REST). À retenir : un jeton a une durée de vie limitée, et doit toujours circuler sur HTTPS.

23.5 CORS

Les navigateurs interdisent par défaut à une page web chargée depuis un site A d'appeler une API située sur un site B. Le serveur doit autoriser explicitement les origines autorisées (*Cross-Origin Resource Sharing*) :

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://mon-frontend.example.com"],  # liste explicite
    allow_methods=["POST", "GET"],
    allow_headers=["*"],
)
```

Attention

`allow\_origins=["*"]` autorise tout le monde. Acceptable pour un prototype local, à éviter en production. CORS ne protège que contre les appels *depuis un navigateur* : ce n'est pas un mécanisme d'authentification.

23.6 Autres bonnes pratiques

- **Valider les entrées** : Pydantic limite les types, les bornes et la taille des lots (`max\_length=100`).
- **Limiter les requêtes** (*rate limiting*) : empêcher un client d'envoyer des milliers de requêtes par seconde ; généralement réglé au niveau du proxy ou d'une bibliothèque dédiée.
- **Ne pas exposer de secrets** dans le code, les logs, les messages d'erreur ou le dépôt Git.
- **Utiliser HTTPS** en production pour chiffrer le trafic.
- **Ne pas charger de fichier modèle de source inconnue** (chapitre 10).

Question de compréhension 23.1

Quelle est la différence entre un **401** et un **403** ? Dans notre API, quand chacun est-il renvoyé ?

Configuration et journalisation

24.1 Variables d'environnement et fichier .env

Écrire un secret directement dans le code est une faute grave :

```
API_KEY = "ma-cle-secrete"          # à NE JAMAIS faire
```

Le code est versionné (Git), partagé, copié. Un secret écrit dedans finit tôt ou tard exposé. De plus, la même application doit tourner avec des valeurs différentes en développement et en production.

La solution : lire la configuration depuis l'**environnement**. En local, un fichier `.env` (non versionné) permet de les définir :

`.env`

```
APP_NAME=Loan Prediction API
API_KEY=une-vraie-cle-longue-et-aleatoire
LOG_LEVEL=INFO
```

On ajoute `.env` au `.gitignore` et on versionne à la place un modèle sans secrets, `.env.example`, qui indique aux autres développeurs quelles variables définir.

24.2 Une configuration propre avec pydantic-settings

`app/config.py`

```
from functools import lru_cache
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

BASE_DIR = Path(__file__).resolve().parent.parent

class Settings(BaseSettings):
    app_name: str = "Loan Prediction API"
    api_key: str = "dev-key"          # valeur par défaut : développement uniquement
    log_level: str = "INFO"
    models_dir: Path = BASE_DIR / "models"

    model_config = SettingsConfigDict(env_file=BASE_DIR / ".env", extra="ignore")

@lru_cache
def get_settings() -> Settings:
```

```
return Settings()
```

- `BaseSettings` lit chaque champ depuis une variable d'environnement du même nom (`API\KEY`), puis depuis `.env`, puis la valeur par défaut ;
- le typage valide la configuration au démarrage (un `LOG_LEVEL` vide est détecté tout de suite) ;
- `lru_cache` garantit que l'objet n'est créé qu'une fois.

Attention

La valeur par défaut "`dev-key`" est pratique pour apprendre, mais dangereuse si elle part en production. En production, la clé doit être définie explicitement et le démarrage doit échouer si elle manque.

24.3 Journalisation (*logging*)

Pourquoi pas `print()` ?

	<code>print()</code>	<code>logging</code>
Niveaux de gravité	Non	DEBUG, INFO, WARNING, ERROR
Horodatage, origine	À ajouter à la main	Intégrés au format
Destination	Sortie standard	Console, fichier, service externe
Filtrage en production	Impossible	Par niveau

Un exemple simple

```
import logging, time

logging.basicConfig(level="INFO",
                    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s")
logger = logging.getLogger("ml_api")

@app.middleware("http")
async def log_requests(request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed_ms = (time.perf_counter() - start) * 1000
    logger.info("%s %s -> %s (%.1f ms)", request.method, request.url.path,
              response.status_code, elapsed_ms)
    return response
```

Un *middleware* est une fonction qui s'exécute autour de *chaque* requête : ici, il mesure la durée et journalise la méthode, le chemin, le code de retour et le temps de réponse.

Que journaliser ?

- les **requêtes** (méthode, chemin, statut, durée) ;
- les **erreurs**, avec la trace (`logger.exception`) ;
- le **chargement du modèle** et sa version ;
- la **version du modèle** utilisée pour chaque prédiction (utile pour comprendre un résultat) ;

Attention

Ne journalisez jamais de secrets (clés, mots de passe) ni de données personnelles complètes.

Question de compréhension 24.1

Pourquoi lit-on la clé d'API depuis l'environnement plutôt que de l'écrire dans `security.py`? Citez deux risques évités.

Performance et conception

Cette section reste volontairement pédagogique : l'objectif est de se forger les bons réflexes, pas de plonger dans l'optimisation avancée.

25.1 Latence et temps de réponse

La **latence** est le temps entre l'envoi de la requête et la réception de la réponse. Elle additionne le réseau, la validation, la transformation, `predict()` et la sérialisation. Mesurez avant d'optimiser : le journal du middleware ci-dessus vous donne la durée côté serveur.

25.2 Charger une fois, prédire souvent

Sur le projet final, mesuré sur une machine ordinaire :

Opération	Durée
Chargement des deux Pipelines (<code>joblib.load</code>)	environ 1 s
Une prédiction de classification, une ligne	environ 3 ms

Charger le modèle à chaque requête multiplierait la latence par plus de 300. Voilà pourquoi on le charge *au démarrage*.

25.3 Concurrency

Plusieurs clients peuvent envoyer des requêtes simultanément. FastAPI exécute les fonctions déclarées avec `def` dans une *pool de threads*, ce qui permet de traiter plusieurs requêtes en parallèle tant que le code libère le *GIL* (c'est le cas de nombreuses opérations NumPy). Une fonction déclarée avec `async def` s'exécute, elle, dans la boucle d'événements : y appeler un `predict()` long *bloque* toutes les autres requêtes. Pour un modèle, la règle prudente est : **déclarer les routes de prédiction avec `def`** (comme dans ce cours).

25.4 Prédiction par lot (*batch*)

Envoyer 100 lignes en une requête est beaucoup plus efficace que 100 requêtes d'une ligne.

Façon de faire	Durée mesurée
100 appels de <code>predict_proba</code> sur 1 ligne	environ 266 ms
1 appel de <code>predict_proba</code> sur 100 lignes	environ 3 ms

Les bibliothèques de Machine Learning sont *vectorisées* : elles traitent un tableau d'un coup. Le projet final expose donc `POST /api/v1/predict/batch`, avec une taille maximale de 100 éléments.

25.5 CPU ou GPU

Les modèles Scikit-learn classiques (régressions, arbres, forêts) tournent sur CPU. Le GPU n'apporte un gain que pour des réseaux de neurones volumineux. Pour un premier service, concentrez-vous sur un modèle correctement chargé et sur le batch avant de chercher du matériel spécialisé.

À retenir

Entraînement lourd, inférence légère : l'API doit servir des prédictions en millisecondes. Chargez une fois, validez tôt, groupez quand c'est possible, mesurez toujours.

Question de compréhension 25.1

Pourquoi une route de prédiction déclarée avec `async def` qui appelle un `predict()` long peut-elle ralentir toute l'API ?

PARTIE XII

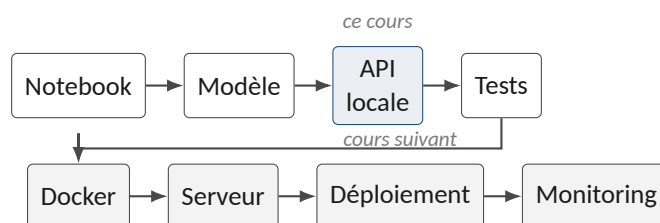
Préparation à la production

Du développement à la production : le chemin

Construire une API n'est pas la mettre en production

Ce chapitre est une *préparation mentale*. Vous n'avez pas besoin de maîtriser Docker, HTTPS ou le cloud pour concevoir une bonne API ; vous devez savoir que ces étapes existent, dans quel ordre, et pourquoi. La production fera l'objet d'un cours dédié.

26.1 Le parcours



26.2 Développement et production

	Développement	Production
Adresse	localhost (127.0.0.1) : vous seul y accédez	Serveur accessible aux clients
Rechargement	--reload	Désactivé
Données	Fictives	Réelles
Secrets	.env local	Gestionnaire de secrets, variables du serveur
Protocole	HTTP	HTTPS
Erreurs	Détaillées, pour déboguer	Génériques, détails dans les logs
Disponibilité	Pas d'enjeu	Doit rester accessible

26.3 Les briques à connaître

Uvicorn et Gunicorn

Uvicorn est le serveur ASGI utilisé jusqu'ici. En production, on lance souvent *plusieurs processus* (*workers*) pour exploiter plusieurs cœurs :

```
$ uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

Gunicorn est un gestionnaire de processus qui peut piloter des workers Uvicorn et les redémarrer en cas de défaillance. Rappel : chaque worker charge sa propre copie du modèle (mémoire × nombre de workers).

Docker

Docker emballe l'application, ses dépendances et sa version de Python dans une *image* qui s'exécute de la même façon partout. Cela règle le classique « ça marche chez moi », y compris le problème des versions de Scikit-learn.

Dockerfile (aperçu)

```
FROM python:3.12-slim
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ app/
COPY models/ models/
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

HTTPS et reverse proxy

HTTPS chiffre les échanges. En pratique, on place devant l'API un **reverse proxy** (Nginx, Caddy, un *load balancer* cloud) qui termine la connexion HTTPS, applique des limites de débit et répartit les requêtes entre plusieurs instances de l'API.



Cloud et variables d'environnement

Un fournisseur cloud (AWS, Azure, Google Cloud, ou des plateformes plus simples) loue des machines ou des conteneurs. La configuration y passe par des *variables d'environnement* et des gestionnaires de secrets : le même code, configuré autrement, sans modification.

Monitoring

Une fois en ligne, on surveille : disponibilité (`/health`), latence, taux d'erreurs, et la *qualité du modèle* elle-même (dérive des données). Un modèle peut se dégrader sans qu'aucune erreur technique n'apparaisse.

À retenir

Le chemin est : *concevoir et tester localement* → *empaqueter* → *déployer* → *surveiller*. Un bon design local (config externalisée, logs, tests, `/health`) rend toutes les étapes suivantes beaucoup plus faciles.

Question de compréhension 26.1

Pourquoi `localhost` suffit-il en développement mais pas en production ? Quel élément s'ajoute pour sécuriser la connexion ?

PARTIE XIII

Projet final complet

Le projet `ml_prediction_api`

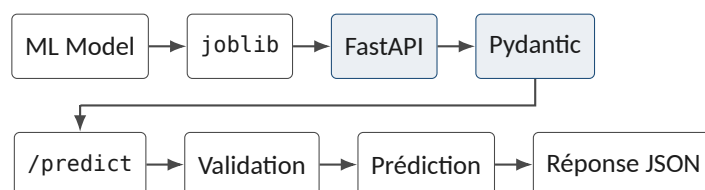
Nous assemblons tout ce qui précède dans un projet réaliste. Le code ci-dessous est *le code réel* du dossier fourni avec ce cours : il a été exécuté et validé par une suite de tests (10 passed).

27.1 Le scénario

Une banque veut intégrer deux modèles à son application :

- un **classifieur** qui prédit si un prêt sera accordé ;
- un **régresseur** qui estime le montant maximal.

Les données d'entraînement sont synthétiques : l'objectif est l'architecture, pas la performance. Les deux modèles sont des **Pipeline** Scikit-learn (imputation, standardisation, *one-hot*, modèle), sauvegardés en `.joblib`.



27.2 Arborescence

```

ml_prediction_api/|—
  app/|
    |— __init__.py|
    |— main.py      # application, lifespan, middleware, erreurs|
    |— config.py    # configuration (variables d'environnement)|
    |— schemas.py   # contrat d'entrée / sortie (Pydantic)|
    |— model_service.py # chargement et appel des Pipelines|
    |— routes.py    # endpoints /api/v1/...|
    |— security.py  # authentification par clé d'API|—
  training/|
    |— train_models.py # produit les .joblib (hors API)|—
  models/      # approval_pipeline.joblib, amount_pipeline.joblib|—
  tests/|
    |— conftest.py|
    |— test_prediction.py|—
  client/|
    |— client.py      # application cliente de démonstration|—
  .env.example|—
  requirements.txt|—
  
```

README.md

27.3 Rôle de chaque fichier

Fichier	Rôle
training/train_models.py	<i>Hors ligne</i> : prépare les données, construit et entraîne les Pipelines, les sauvegarde avec leurs métadonnées
app/config.py	Lit la configuration (clé, niveau de logs, dossier des modèles)
app/schemas.py	Le contrat : <code>LoanRequest</code> en entrée, réponses typées
app/model_service.py	Charge les Pipelines une fois, reconstruit le <code>DataFrame</code> , appelle <code>predict</code> , convertit les types
app/security.py	Vérifie la clé d'API; 401 / 403
app/routes.py	Trois endpoints : <code>/predict</code> , <code>/predict/amount</code> , <code>/predict/batch</code>
app/main.py	Assemble : lifespan, middleware de logs, gestionnaires d'erreurs, <code>/health</code>
tests/	Vérifient le contrat (succès, validations, accès)
client/client.py	Consomme l'API comme le ferait une autre application

27.4 L'entraînement (côté Data Scientist)

Ce script n'est *pas* exécuté par l'API. Il produit les fichiers qu'elle charge.

training/train_models.py

```

"""Entraînement (côté Data Scientist) : produit les Pipelines sauvegardés en .joblib.

Ce script n'est PAS exécuté par l'API : il tourne une fois, hors ligne.
"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

OUT_DIR = Path(__file__).resolve().parent.parent / "models"
NUM = ["age", "income", "years_employed", "debt_ratio"]
CAT = ["education", "employment_type"]
FEATURES = ["age", "income", "education", "employment_type", "years_employed", "

```

```

→debt_ratio"]

def make_data(n: int = 3000, seed: int = 42) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    df = pd.DataFrame({
        "age": rng.integers(18, 70, n),
        "income": rng.normal(3_000_000, 1_200_000, n).clip(300_000, None),
        "education": rng.choice(["secondaire", "licence", "master", "doctorat"], n,
                                p=[0.35, 0.4, 0.2, 0.05]),
        "employment_type": rng.choice(["salarie", "independant", "sans_emploi"], n,
                                       p=[0.6, 0.25, 0.15]),
        "years_employed": rng.uniform(0, 30, n),
        "debt_ratio": rng.uniform(0, 0.9, n),
    })
    df.loc[rng.random(n) < 0.1, "years_employed"] = np.nan # valeurs manquantes
    edu_bonus = df["education"].map({"secondaire": 0, "licence": 0.4, "master": 0.8, "
→doctorat": 1.0})
    emp_pen = df["employment_type"].map({"salarie": 0, "independant": -0.3, "sans_emploi
→": -1.5})
    score = (df["income"] / 1_500_000 - 2.0 + edu_bonus + emp_pen
              - 4.0 * df["debt_ratio"] + 0.05 * df["years_employed"].fillna(5))
    df["approved"] = (score + rng.normal(0, 0.5, n) > 0).astype(int)
    df["max_amount"] = (df["income"] * (2.5 - 2.0 * df["debt_ratio"]) * (1 + 0.1 *
→edu_bonus)
                      * (emp_pen.eq(-1.5).map({True: 0.3, False: 1.0})))
                      + rng.normal(0, 150_000, n)).clip(0, None)

    return df

def preprocessor() -> ColumnTransformer:
    return ColumnTransformer([
        ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
                          ("scaler", StandardScaler())])), (NUM),
        ("cat", OneHotEncoder(handle_unknown="ignore"), CAT),
    ])

def main() -> None:
    OUT_DIR.mkdir(exist_ok=True)
    df = make_data()
    X = df[FEATURES]

    Xtr, Xte, ytr, yte = train_test_split(X, df["approved"], test_size=0.2,
                                         random_state=42, stratify=df["approved"])
    clf = Pipeline([("preprocessor", preprocessor()),
                    ("model", LogisticRegression(max_iter=1000))]).fit(Xtr, ytr)
    print("Accuracy classification :", round(accuracy_score(yte, clf.predict(Xte)), 3))

    Xtr, Xte, ytr, yte = train_test_split(X, df["max_amount"], test_size=0.2,
→random_state=42)
    reg = Pipeline([("preprocessor", preprocessor()),
                    ("model", RandomForestRegressor(n_estimators=80, random_state=42,

```

```

→n_jobs=1))])
        ).fit(Xtr, ytr)
    print("MAE régression          :", round(mean_absolute_error(yte, reg.predict(Xte)))
→)

    joblib.dump(clf, OUT_DIR / "approval_pipeline.joblib")
    joblib.dump(reg, OUT_DIR / "amount_pipeline.joblib")
    (OUT_DIR / "metadata.json").write_text(json.dumps({
        "version": "1.0.0",
        "sklearn_version": sklearn.__version__,
        "features": FEATURES,
    }, indent=2))
    print("Modèles sauvegardés dans", OUT_DIR)

if __name__ == "__main__":
    main()

```

27.5 La configuration

app/config.py

```

"""Configuration de l'application, lue depuis l'environnement / le fichier .env."""
from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

BASE_DIR = Path(__file__).resolve().parent.parent

class Settings(BaseSettings):
    app_name: str = "Loan Prediction API"
    api_key: str = "dev-key" # valeur par défaut : développement uniquement
    log_level: str = "INFO"
    models_dir: Path = BASE_DIR / "models"

    model_config = SettingsConfigDict(env_file=BASE_DIR / ".env", extra="ignore")

@lru_cache
def get_settings() -> Settings:
    """Crée l'objet Settings une seule fois (mis en cache)."""
    return Settings()

```

27.6 Le contrat : schemas.py

app/schemas.py

```

"""Schémas Pydantic : le « contrat » de l'API (entrées et sorties)."""
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field

class LoanRequest(BaseModel):
    """Données d'un client envoyées par l'application cliente."""

    age: int = Field(..., ge=18, le=100, description="Âge en années")
    income: float = Field(..., ge=0, description="Revenu annuel")
    education: Literal["secondaire", "licence", "master", "doctorat"]
    employment_type: Literal["salarie", "independant", "sans_emploi"]
    years_employed: float | None = Field(
        default=None, ge=0, le=60, description="Ancienneté (optionnelle)"
    )
    debt_ratio: float = Field(..., ge=0, le=1, description="Taux d'endettement (0 à 1)")

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "age": 35,
                "income": 4200000,
                "education": "licence",
                "employment_type": "salarie",
                "years_employed": 6.5,
                "debt_ratio": 0.28,
            }
        }
    )

class ApprovalResponse(BaseModel):
    prediction: int = Field(..., description="1 = prêt accordé, 0 = refusé")
    label: str
    probability: float = Field(..., ge=0, le=1)
    model_version: str

class AmountResponse(BaseModel):
    predicted_amount: float = Field(..., description="Montant maximal estimé")
    model_version: str

class BatchRequest(BaseModel):
    items: list[LoanRequest] = Field(..., min_length=1, max_length=100)

class BatchResponse(BaseModel):
    predictions: list[ApprovalResponse]
    count: int

```

```
class ErrorResponse(BaseModel):
    detail: str
```

À retenir

`Literal[...]` restreint les catégories aux valeurs connues du modèle ; `years\employed` est optionnel parce que le Pipeline sait imputer la médiane ; `json\schema\extra` alimente l'exemple de Swagger.

27.7 Le service ML : model_service.py

app/model_service.py

```
"""Couche « service » : tout ce qui touche au modèle est ici, pas dans les routes."""
import json
import logging
from pathlib import Path

import joblib
import pandas as pd

from app.schemas import LoanRequest

logger = logging.getLogger("ml_api.model_service")

FEATURES = [
    "age", "income", "education", "employment_type", "years_employed", "debt_ratio",
]

class ModelNotLoadedError(RuntimeError):
    """Le modèle n'a pas pu être chargé (fichier absent, etc.)."""

class PredictionError(RuntimeError):
    """Erreur survenue pendant predict()."""

class ModelService:
    def __init__(self, models_dir: Path):
        self.models_dir = Path(models_dir)
        self.classifier = None
        self.regressor = None
        self.version = "unknown"

    def load(self) -> None:
        """Charge les pipelines UNE SEULE FOIS (au démarrage de l'application)."""
        try:
            self.classifier = joblib.load(self.models_dir / "approval_pipeline.joblib")
```

```

        self.regressor = joblib.load(self.models_dir / "amount_pipeline.joblib")
        meta = json.loads((self.models_dir / "metadata.json").read_text())
        self.version = meta.get("version", "unknown")
    except FileNotFoundError as exc:
        raise ModelNotLoadedError(f"Fichier modèle introuvable : {exc.filename}")
→ from exc
        logger.info("Modèles chargés (version %s)", self.version)

@property
def is_ready(self) -> bool:
    return self.classifier is not None and self.regressor is not None

@staticmethod
def _to_dataframe(items: list[LoanRequest]) -> pd.DataFrame:
    """Reconstruit exactement les colonnes (et leur ordre) vues à l'entraînement."""
    rows = [item.model_dump() for item in items]
    return pd.DataFrame(rows, columns=FEATURES)

def predict_approval(self, items: list[LoanRequest]) -> list[dict]:
    if not self.is_ready:
        raise ModelNotLoadedError("Modèle non chargé")
    df = self._to_dataframe(items)
    try:
        classes = self.classifier.predict(df)
        probas = self.classifier.predict_proba(df)[: , 1]
    except Exception as exc:  # erreur interne du pipeline
        logger.exception("Échec de predict()")
        raise PredictionError("Erreur interne pendant la prédiction") from exc
    return [
        {
            "prediction": int(c),
            "label": "accordé" if c == 1 else "refusé",
            "probability": round(float(p), 4),
            "model_version": self.version,
        }
        for c, p in zip(classes, probas)
    ]

def predict_amount(self, item: LoanRequest) -> dict:
    if not self.is_ready:
        raise ModelNotLoadedError("Modèle non chargé")
    df = self._to_dataframe([item])
    try:
        value = float(self.regressor.predict(df)[0])
    except Exception as exc:
        logger.exception("Échec de predict()")
        raise PredictionError("Erreur interne pendant la prédiction") from exc
    return {"predicted_amount": round(value, 2), "model_version": self.version}

```

- `load()` est appelé *une fois* au démarrage ; les Pipelines restent en mémoire ;
- `\_to\_dataframe` impose les colonnes et leur ordre (`FEATURES`) ;

- les erreurs internes sont *journalisées* puis converties en `PredictionError`;
- les résultats sont convertis en types Python (`int`, `float`) avant la sérialisation JSON;
- ce fichier ne connaît ni HTTP ni FastAPI : on pourrait le tester seul.

27.8 Sécurité et routes

app/security.py

```
"""Authentification simple par clé d'API (en-tête X-API-Key)."""
import secrets

from fastapi import Depends, HTTPException, status
from fastapi.security import APIKeyHeader

from app.config import Settings, get_settings

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def verify_api_key(
    key: str | None = Depends(api_key_header),
    settings: Settings = Depends(get_settings),
) -> None:
    if key is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Clé d'API manquante")
    if not secrets.compare_digest(key, settings.api_key):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Clé d'API invalide")
```

app/routes.py

```
"""Routes de prédiction (version 1 de l'API)."""
from fastapi import APIRouter, Depends, Request

from app.model_service import ModelService
from app.schemas import (
    AmountResponse, ApprovalResponse, BatchRequest, BatchResponse,
    ErrorResponse, LoanRequest,
)
from app.security import verify_api_key

router = APIRouter(
    prefix="/api/v1",
    tags=["prédiction"],
    dependencies=[Depends(verify_api_key)],
    responses={
        401: {"model": ErrorResponse, "description": "Clé d'API manquante"},
        403: {"model": ErrorResponse, "description": "Clé d'API invalide"},
        503: {"model": ErrorResponse, "description": "Modèle indisponible"},
    },
)
```

```

def get_service(request: Request) -> ModelService:
    """Dépendance : récupère le service chargé au démarrage."""
    return request.app.state.model_service

@router.post("/predict", response_model=ApprovalResponse, summary="Décision d'octroi de
→prêt")
def predict(payload: LoanRequest, service: ModelService = Depends(get_service)):
    """Reçoit les données d'un client et renvoie la décision du modèle de classification.
    →"""
    return service.predict_approval([payload])[0]

@router.post("/predict/amount", response_model=AmountResponse, summary="Montant maximal
→estimé")
def predict_amount(payload: LoanRequest, service: ModelService = Depends(get_service)):
    """Second modèle (régression) exposé par la même API."""
    return service.predict_amount(payload)

@router.post("/predict/batch", response_model=BatchResponse, summary="Prédictions en lot
→")
def predict_batch(payload: BatchRequest, service: ModelService = Depends(get_service)):
    results = service.predict_approval(payload.items)
    return {"predictions": results, "count": len(results)}

```

`dependencies=[Depends(verify\_api\_key)]` sur le routeur protège *toutes* les routes de prédiction d'un coup, sans répéter le code. Les routes sont courtes : elles reçoivent un objet validé, délèguent au service et renvoient un résultat typé.

27.9 L'application : main.py

app/main.py

```

"""Point d'entrée : crée l'application, charge les modèles, branche les routes."""
import logging
import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from app.config import get_settings
from app.model_service import ModelNotLoadedError, ModelService, PredictionError
from app.routes import router

settings = get_settings()
logging.basicConfig(
    level=settings.log_level,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",

```

```

)
logger = logging.getLogger("ml_api")

@asynccontextmanager
async def lifespan(app: FastAPI):
    # --- Démarrage : exécuté une seule fois ---
    service = ModelService(settings.models_dir)
    try:
        service.load()
    except ModelNotLoadedError:
        logger.exception("Démarrage sans modèle : /health indiquera 'degraded'")
    app.state.model_service = service
    yield
    # --- Arrêt : libération éventuelle de ressources ---
    logger.info("Arrêt de l'application")

app = FastAPI(
    title=settings.app_name,
    description="API de prédiction : octroi de prêt (classification) et montant (
→régression).",
    version="1.0.0",
    lifespan=lifespan,
)
app.include_router(router)

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed_ms = (time.perf_counter() - start) * 1000
    logger.info("%s %s -> %s (%.1f ms)", request.method, request.url.path,
                response.status_code, elapsed_ms)
    return response

@app.exception_handler(ModelNotLoadedError)
async def model_not_loaded_handler(request: Request, exc: ModelNotLoadedError):
    return JSONResponse(status_code=503, content={"detail": "Modèle indisponible"})

@app.exception_handler(PredictionError)
async def prediction_error_handler(request: Request, exc: PredictionError):
    return JSONResponse(status_code=500, content={"detail": str(exc)})

@app.get("/", tags=["santé"])
def home():
    return {"message": "API opérationnelle", "docs": "/docs"}

```

```
@app.get("/health", tags=["santé"])
def health(request: Request):
    ready = request.app.state.model_service.is_ready
    return {"status": "ok" if ready else "degraded", "model_loaded": ready}
```

Remarquez la *tolérance au démarrage* : si le modèle manque, l'application démarre quand même, /health annonce **degraded** et les routes de prédiction répondent **503**. Un orchestrateur peut ainsi détecter le problème sans que le serveur disparaisse.

27.10 Les tests

tests/test_prediction.py

```
def test_home(client):
    r = client.get("/")
    assert r.status_code == 200
    assert r.json()["message"] == "API opérationnelle"

def test_health(client):
    assert client.get("/health").json() == {"status": "ok", "model_loaded": True}

def test_predict_ok(client, headers, valid_payload):
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 200
    body = r.json()
    assert body["prediction"] in (0, 1)
    assert 0.0 <= body["probability"] <= 1.0

def test_predict_missing_optional_field(client, headers, valid_payload):
    valid_payload.pop("years_employed")  # valeur manquante gérée par l'imputer
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 200

def test_validation_error_422(client, headers, valid_payload):
    valid_payload["age"] = 5  # < 18
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 422

def test_unknown_category_422(client, headers, valid_payload):
    valid_payload["education"] = "astronaute"
    r = client.post("/api/v1/predict", json=valid_payload, headers=headers)
    assert r.status_code == 422

def test_missing_api_key_401(client, valid_payload):
    assert client.post("/api/v1/predict", json=valid_payload).status_code == 401
```

```
def test_wrong_api_key_403(client, valid_payload):
    r = client.post("/api/v1/predict", json=valid_payload, headers={"X-API-Key": "nope"}
    →))
    assert r.status_code == 403

def test_amount(client, headers, valid_payload):
    r = client.post("/api/v1/predict/amount", json=valid_payload, headers=headers)
    assert r.status_code == 200
    assert r.json()["predicted_amount"] > 0

def test_batch(client, headers, valid_payload):
    r = client.post("/api/v1/predict/batch", json={"items": [valid_payload] * 3},
    →headers=headers)
    assert r.status_code == 200
    assert r.json()["count"] == 3
```

27.11 Lancer le projet

```
$ cd ml_prediction_api
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
$ cp .env.example .env                # puis éditer API_KEY
$ python training/train_models.py     # crée les fichiers .joblib
$ pytest -q                          # 10 passed
$ uvicorn app.main:app --reload       # http://127.0.0.1:8000/docs
$ python client/client.py             # dans un autre terminal
```

Si vous définissez `API\_KEY` dans `.env`, exportez la même valeur avant de lancer le client (`export API\_KEY=...`) ou les appels seront refusés avec un 403.

27.12 Réponse type

```
{"prediction": 1, "label": "accordé", "probability": 0.9631, "model_version": "1.0.0"}
```

Exercices

Exercice 13 • Niveau 10 — Projet final

Objectif : appliquer toute la démarche à *votre* modèle. **Consignes** : prenez un modèle de classification que vous avez entraîné (ou construisez-en un sur un jeu de données de votre

choix) et réalisez : (1) un **Pipeline** avec prétraitement ; (2) **joblib** avec métadonnées ; (3) **POST /api/v1/predict** avec schéma Pydantic et contraintes ; (4) clé d'API ; (5) gestion des erreurs 422, 503 et 500 ; (6) quatre tests **pytest** ; (7) un client Python ; (8) un **README.md**. **Résultat attendu** : un dépôt démarrable en cinq commandes, dont les tests passent.

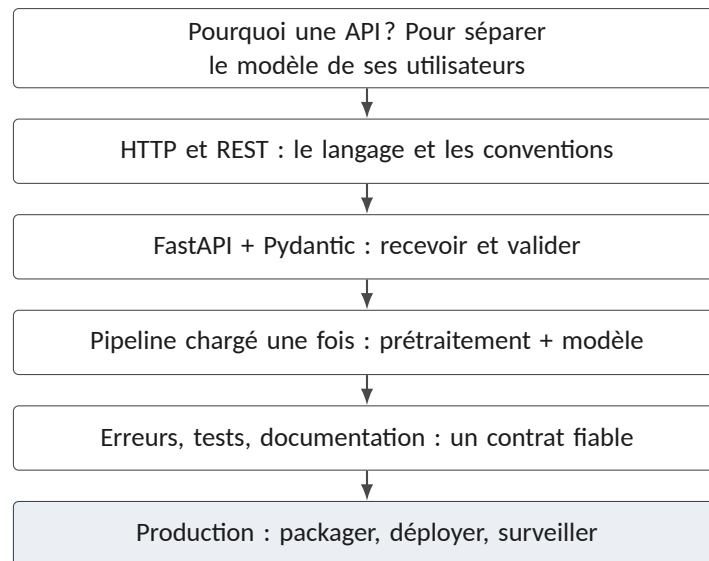
Correction

Correction — Projet final

Il n'y a pas de correction unique : le projet de ce chapitre en est une. Vérifiez-vous avec la checklist finale. Les erreurs les plus courantes sont : prétraitement hors du Pipeline, modèle chargé dans la route, `numpy.int64` renvoyé tel quel, secrets écrits dans le code, tests absents pour les cas d'erreur.

Conclusion

Vous êtes parti d'un fichier `model.joblib` et d'une question : comment une autre application peut-elle l'utiliser ? Vous savez maintenant que la réponse n'est pas « FastAPI », mais une chaîne de raisonnements :



Pour aller plus loin : approfondir la sécurité (OAuth2, rôles), la persistance (SQLAlchemy), la conteneurisation (Docker, Docker Compose), le déploiement (cloud, intégration continue) et le suivi des modèles (versionnage, dérive, outils de MLOps).

Checklist finale

Avant de déclarer qu'une API de prédiction est « propre », vérifiez :

Le modèle

- ☐ Le Pipeline complet (prétraitement + modèle) est sauvegardé, pas seulement le modèle.
- ☐ Les métadonnées (version du modèle, version de Scikit-learn, liste des variables) sont enregistrées.
- ☐ Le fichier ne provient que d'une source de confiance.
- ☐ Le modèle est chargé *une fois*, au démarrage.

Le contrat

- ☐ Un schéma Pydantic décrit les entrées, avec types, bornes et catégories (`Literal`).
- ☐ Un modèle de réponse décrit la sortie ; les types NumPy sont convertis.
- ☐ L'endpoint est `POST` avec un nom clair et un préfixe de version (`/api/v1/...`).
- ☐ La documentation (`/docs`) contient exemples et descriptions.

La robustesse

- ☐ Les codes `401`, `403`, `422`, `503`, `500` sont gérés avec des messages propres.
- ☐ Les erreurs internes sont journalisées mais pas exposées au client.
- ☐ `/health` indique si le modèle est chargé.

La qualité du projet

- ☐ Le code est séparé : schémas, service, routes, configuration.
- ☐ Aucun secret dans le code ; `.env` ignoré par Git ; `.env.example` fourni.
- ☐ Des tests `pytest` couvrent succès, validation, accès et erreurs.
- ☐ Les logs contiennent méthode, chemin, statut et durée.
- ☐ Un `README.md` explique comment installer et lancer.

La préparation à la production

- ☐ Je sais expliquer la différence entre `localhost` et un serveur accessible.
- ☐ Je sais à quoi servent Uvicorn, Docker, HTTPS et un reverse proxy.
- ☐ Je sais ce qu'il faudrait surveiller une fois l'API en ligne.

Aide-mémoire

Je veux...	J'utilise...
Lancer l'API en développement	<code>uvicorn app.main:app --reload</code>
Voir la documentation	<code>/docs</code> (Swagger), <code>/redoc</code>
Envoyer des données à un modèle	<code>POST</code> avec corps JSON
Valider les entrées	Modèle Pydantic, <code>Field(...)</code> , <code>Literal</code>
Charger le modèle une fois	<code>lifespan</code> + <code>app.state</code>
Reproduire le prétraitement	<code>Pipeline</code> + <code>ColumnTransformer</code> sauvegardé
Renvoyer une erreur volontaire	<code>raise HTTPException(status_code=..., detail=...)</code>
Protéger les routes	<code>Depends(verify_api_key)</code>
Lire la configuration	<code>pydantic-settings</code> + <code>.env</code>
Tester sans serveur	<code>TestClient(app)</code> + <code>pytest</code>
Remplacer le modèle dans un test	<code>app.dependency_overrides</code>
Prédire plusieurs lignes à la fois	Endpoint <code>/predict/batch</code>